

Async & Live Trading — เทรดจริงแบบอัตโนมัติ

บทที่ 31–35: Async Fundamentals · asyncio Patterns · WebSocket & Live Data · Order Execution · Live System Integration
จบ Part นี้ → ระบบ live trading อัตโนมัติที่รับ feed จริง ส่งออเดอร์จริง พร้อม kill switch และ monitoring

📖 ตัวเลขใน ▶ OUTPUT ของ Part นี้คืออะไร

ต่างจาก Part อื่นตรงที่โค้ดใน Part นี้ **คุยกับโลกภายนอก** — ราคาจากตลาด เวลาที่ข้อความมาถึง ความเร็วเน็ต · ผลลัพธ์จึง **ไม่มีทางเหมือนกันสองครั้ง** แม้รันบนเครื่องเดียวกัน

ดังนั้นตัวเลขราคา · timestamp · จำนวนข้อความที่ได้รับ ในกล่อง OUTPUT ของ Part นี้เป็น **ตัวอย่างรูปแบบ** ให้ดูว่า "หน้าตาผลลัพธ์ควรเป็นแบบไหน" ไม่ใช่ค่าที่คุณต้องได้ตรงกัน · **สิ่งที่ต้องตรงคือโครงสร้าง** — มีฟิลด์ครบไหม ลำดับเหตุการณ์ถูกไหม error ถูกจัดการไหม

🔗 ถ้าอยากทดสอบโค้ด Part นี้โดยไม่ต้องเน็ตจริง: เขียน mock ที่คืนข้อความรูปแบบเดียวกับ exchange แล้วป้อนเข้าฟังก์ชัน parse — เป็นวิธีเดียวกับที่บทที่ 28 สอนเรื่องแยก "โค้ดดึงข้อมูล" ออกจาก "โค้ดคำนวณ"

🔗 อ่าน Part นี้ยังไ

🟢 **เพิ่งเคยเจอ async ครั้งแรก** — อ่านบทที่ 31 ให้ช้าที่สุดใจในเล่ม โดยเฉพาะกล่อง  ที่แกะ async def / await / gather · 4 คำนี้คือรากของทั้ง Part ถ้าข้ามไป บทที่ 32-35 จะกลายเป็นการลอกโค้ด · เจอบล็อกโค้ดยาว ๆ ให้หากล่อง  "โครงก่อนดูโค้ด" ที่อยู่นั่นก่อนเสมอ แล้วค่อยลงอ่านรายละเอียด

🟢 **เขียน async เป็นแล้ว** — กวาดบทที่ 31-32 ผ่านได้ · ของจริงสำหรับคุณอยู่ในกล่อง  **[รอบสอง]** และ ⚠ กับดัก: thundering herd และ jitter, เหตุผลที่ไม่ดี Exception เปลา, gather กับ exception ที่ทั้ง task ค้าง, backpressure ของ asyncio.Queue , และเส้นแบ่ง I/O-bound vs CPU-bound ที่ async ช่วยไม่ได้

▼ สารบัญ Part นี้ — 6 บท · 21 หัวข้อ

บทที่ 31: Async Fundamentals

31.1 ปัญหาของ Blocking Code · 31.2 async def และ await — หัวใจของ Async · 31.3 Event Loop — หัวใจของ asyncio · 31.4 ทำไม Blocking Code ถึงอันตรายใน Async

บทที่ 32: asyncio Patterns

32.1 asyncio.gather() — รันพร้อมกัน · 32.2 asyncio.Queue — Decouple Producer และ Consumer · 32.3 asyncio.wait_for() — Timeout · 32.4 asyncio.Task — Background Jobs และ Cancellation · 32.5 asyncio.Semaphore — จำกัดจำนวน Concurrent Requests

บทที่ 33: WebSocket & Live Data

33.1 Binance Futures WebSocket — รูปแบบ Message จริง · 33.2 Bybit WebSocket — รูปแบบ Message · 33.3 WebSocket Client ที่ Reconnect อัตโนมัติ · 33.4 Dual-Feed WebSocket — รับ Binance + Bybit พร้อมกัน

บทที่ 34: Order Execution

34.1 Order State Machine · 34.2 REST API Wrapper พร้อม Rate Limiting และ Retry · 34.3 ทดสอบบน Testnet

บทที่ 35: Live System Integration

35.1 Structured Logging · 35.2 Kill Switch ด้วย asyncio.Event · 35.3 Health Check Loop · 35.4 ระบบ Live Trading ครบวงจร · 35.5 Pre-Flight Checklist ก่อน Go Live

จบหนังสือ — เส้นทางต่อไป

ทำไม PART นี้ถึงสำคัญที่สุด?

Backtest บอกว่า strategy ทำกำไรได้ — แต่นั่นคือบน *ข้อมูลอดีต* ที่นิ่งอยู่ใน DataFrame
ในโลกจริง ราคาเปลี่ยนทุกมิลลิวินาที มีหลาย feed พร้อมกัน ต้องส่ง order ตรงเวลา ต้องจัดการ error ที่ไม่คาดคิด

- **Async** ให้ Python จัดการงานหลายอย่างพร้อมกันโดยไม่ต้องใช้ thread จำนวนมาก
- **WebSocket** รับ market data แบบ real-time แทน polling ทุก 1 วินาที
- **Order Execution** ส่ง order อย่างปลอดภัย มี rate limit และ retry
- **Kill Switch** หยุดระบบทันทีเมื่อเกิดเหตุฉุกเฉิน

🛑 STOP — อ่านก่อนเริ่ม Part VI: Production Safety Checklist

Part VI สอน async และ live trading กับเงินจริง ก่อนเดินหน้าให้ตอบ YES ทุกข้อ:

1.  **API Keys อยู่ใน .env file** — ห้าม hardcode ใน code เด็ดขาด
2.  **ทดสอบบน Testnet ครบ 48+ ชั่วโมง** — Bybit Testnet / Binance Testnet
3.  **Kill switch พร้อม** — มีปุ่มหรือ Telegram command หยุดระบบทันที

4. **✓ Drawdown limit** ตั้งไว้แล้ว — เช่น หยุดอัตโนมัติเมื่อขาดทุน 5%
5. **✓ Position size** ไม่เกิน 5% ของทุน ต่อ 1 trade สำหรับ system ใหม่
6. **✓ Log ทุก order** — timestamp, price, size, status เก็บไว้อย่างน้อย 30 วัน
7. **✓ Monitor** ตลอด 24 ชั่วโมงแรก — อย่าปล่อยให้ระบบทำงานโดยไม่มีคนดู

ถ้ายังไม่ครบ — ทำ Part VI ต่อเพื่อเรียนรู้ได้ แต่อย่าเชื่อมกับ real account จนกว่าจะผ่านทุกข้อ

Running Example ตลอด Part VI — BTC Pair Strategy แบบ Live

เราจะนำ PairStrategy จาก Part II/III มาเปลี่ยนให้รันแบบ real-time:

บท 31 → เข้าใจ async · บท 32 → asyncio patterns · บท 33 → WebSocket feed

บท 34 → ส่ง order ผ่าน REST API · บท 35 → ระบบครบวงจรพร้อม deploy

คำเตือนสำคัญก่อนเริ่ม Part VI

- ทดสอบบน Testnet เสมอ — ยำนำ code จาก tutorial ไปรันบน real account โดยตรง
- ห้าม hardcode API Key — ใช้ environment variable หรือ secrets manager เท่านั้น
- ต้องมี Kill Switch — ทุกระบบ live ต้องหยุดได้ทันทีเมื่อกด Ctrl+C หรือรับ signal
- Log ทุก order — ถ้าไม่มี log จะตามสอบไม่ได้ว่าเกิดอะไรขึ้น
- Monitor drawdown — ตั้ง max drawdown threshold ให้ระบบหยุดอัตโนมัติ

บทที่ 31: Async Fundamentals

แก่นของบทนี้

Async คือวิธีให้ Python "รอ" อะไรบางอย่าง (เช่น network) โดยไม่บล็อก thread — ทำให้ทำงานหลายอย่างพร้อมกันใน thread เดียว เหมือนพนักงานร้านอาหาร 1 คนที่รับออเดอร์หลายโต๊ะพร้อมกัน แทนที่จะยืนรอโต๊ะแรกก่อนแล้วค่อยไปโต๊ะถัดไป

31.1 ปัญหาของ Blocking Code

ใน live trading เราต้องรับราคาจากหลาย exchange พร้อมกัน หาก code ทำงานแบบ blocking จะช้ามาก

```
# =====
# แบบ BLOCKING — รอทีละตัว (ช้า)
# =====

import time
import requests

def fetch_price_blocking(exchange: str) -> float:
    # จำลองการเรียก API (ใช้เวลา 1 วินาที)
    time.sleep(1)
    prices = {"binance": 65000.0, "bybit": 65010.0,
              "okx": 64990.0, "bitget": 65005.0}
    return prices[exchange]

def get_all_prices_blocking():
    start = time.time()
    results = {}
    for ex in ["binance", "bybit", "okx", "bitget"]:
        results[ex] = fetch_price_blocking(ex) # รอทีละตัว!
    elapsed = time.time() - start
    print(f"Blocking: ใช้เวลา {elapsed:.1f}s")
    return results

get_all_prices_blocking()
```

► OUTPUT

Blocking: ใช้เวลา 4.0s

```
# =====
# แบบ ASYNC — รอพร้อมกัน (เร็ว)
# =====

import asyncio
import time

async def fetch_price_async(exchange: str) -> float:
    # asyncio.sleep ไม่บล็อก event loop
    await asyncio.sleep(1)
    prices = {"binance": 65000.0, "bybit": 65010.0,
              "okx": 64990.0, "bitget": 65005.0}
    return prices[exchange]

async def get_all_prices_async():
    start = time.time()
    # gather รัน coroutine ทั้งหมดพร้อมกัน
    results = await asyncio.gather(
        fetch_price_async("binance"),
        fetch_price_async("bybit"),
        fetch_price_async("okx"),
        fetch_price_async("bitget"),
    )
    elapsed = time.time() - start
    print(f"Async: ใช้เวลา {elapsed:.1f}s")
    exchanges = ["binance", "bybit", "okx", "bitget"]
    return dict(zip(exchanges, results))
```

```
# รัน event loop
prices = asyncio.run(get_all_prices_async())
print(prices)
```

► OUTPUT

```
Async: ใช้เวลา 1.0s
{'binance': 65000.0, 'bybit': 65010.0, 'okx': 64990.0, 'bitget': 65005.0}
```

🔴**อ่านว่า:** 4.0s → 1.0s **ทั้งที่ยังรอ 1 วินาทีเท่าเดิมทุกตลาด** — เพราะเปลี่ยนจาก "รอทีละตัวต่อกัน" เป็น "เริ่มรอทั้ง 4 ตัวพร้อมกันแล้วรอครั้งเดียว" · เวลารวมจึงเท่ากับตัวที่ช้าที่สุด ไม่ใช่ผลรวม

📺 3 คำใหม่ที่เพิ่งโผล่พร้อมกัน — async, await, gather

```
async def fetch_price_async(exchange): # ❶ async def
    await asyncio.sleep(1)             # ❷ await
    ...

results = await asyncio.gather(        # ❸ gather
    fetch_price_async("binance"),
    fetch_price_async("bybit"),
)

prices = asyncio.run(get_all_prices_async()) # ❹ run
```

ทั้ง 4 ตัวนี้ทำงานร่วมกันเป็นชุดเดียว อ่านตามลำดับนี้:

1. `async def` — "ฟังก์ชันนี้หยุดกลางคันได้" · เรียกแล้ว **ยังไม่ทำงานทันที** แต่คืน "ใบสั่งงาน" (coroutine) กลับมา — ต่างจาก `def` ปกติที่เรียกแล้วทำเลยจนจบ
2. `await` — "ตรงนี้ต้องรอ · ระหว่างรอ ปลอมให้คนอื่นทำงานไปก่อน" · จุดที่มี `await` คือจุดเดียวที่งานอื่นแทรกได้ ถ้าไม่มี `await` เลย `async` ก็ไม่ต่างจากโค้ดปกติ
3. `asyncio.gather(a, b, c)` — "เอาใบสั่งงานทั้งหมดนี้ไปเริ่มพร้อมกัน แล้วรอจนครบทุกใบ" · คืนผลลัพธ์เป็น list เรียงตามลำดับที่ใส่เข้าไป ไม่ใช่ตามลำดับที่เสร็จ
4. `asyncio.run(...)` — "เปิดสวิตช์ event loop แล้ววิ่งใบสั่งงานนี้จนจบ" · เป็นสะพานจากโลกปกติเข้าโลก `async` เรียกได้จากโค้ดธรรมดาเท่านั้น (ห้ามเรียกซ้อนใน `async def`)

จำง่าย: `async def` = เขียนใบสั่งงาน · `await` = จุดพัก · `gather` = เริ่มพร้อมกัน · `run` = เปิดสวิตช์

📺 ถ้ารู้สึกวุ่นวายกับฟังก์ชันที่หยุดค้างไว้แล้วไปต่อได้" คำนึงว่า มันคือแนวคิดเดียวกับ `yield` ในหัวข้อ 7.7 (Part I) · generator หยุดรอคนมาขอของชิ้นถัดไป · coroutine หยุดรองาน I/O ที่ยังไม่เสร็จ · กลไกเบื้องหลังคือตัวเดียวกัน ซึ่งเป็นเหตุผลที่ `async for` มีอยู่ได้

⚠️ กับดักที่ทำให้ `async` ไม่เร็วขึ้นเลย — `time.sleep` ปนใน `async def`

ถ้าผมเขียน `time.sleep(1)` แทน `await asyncio.sleep(1)` ข้างใน `async def` โค้ดจะยังรันได้ **ไม่ error** แต่กลับไปช้า 4 วินาทีเหมือนเดิม เพราะ `time.sleep` **แซ่แซ่ event loop ทั้งวง** — งานอื่นแทรกไม่ได้เลย

🔴**กฎ:** ข้างใน `async def` ทุกการรอต้องมี `await` นำหน้า ถ้าเจอฟังก์ชันรอที่ไม่มี `await` ได้ (เช่น `requests.get`, `time.sleep`, การอ่านไฟล์ใหญ่) แปลว่ามันจะบล็อกทั้งระบบ

🔴 [รอบสอง] `async` ไม่ได้ทำให้ทุกอย่างเร็วขึ้น

`async` เร่งได้เฉพาะงานที่ "รออย่างอื่นอยู่" (I/O-bound: network, disk) เพราะมันแค่เอาเวลารอว่างระหว่างรอไปทำงานอื่น · งานที่ **CPU ทำงานเต็มที่** (I/O ไม่มี — เช่น คำนวณ backtest ล้านรอบ, optimize portfolio) `async` **ไม่ช่วยเลยแม้แต่นิดเดียว** เพราะไม่มีช่วง "ว่างระหว่างรอ" ให้แทรก — กรณีนั้นต้องใช้ `multiprocessing`

ในสาย quant เส้นแบ่งนี้ชัดมาก: **ดึงราคาจาก 10 ตลาด** = `async` ช่วยเต็ม ๆ · **รัน Monte Carlo 100k paths** = `async` เปล่าประโยชน์

อีกจุดที่มือโปรพลาดบ่อย: `gather` ถ้ามีตัวใดตัวหนึ่ง raise exception ตัวที่เหลือยังวิ่งต่อไปเบื้องหลัง และ exception แรกจะเด็นออกมาทันที — ถ้าอยากได้ผลทุกตัวรวม error ต้องใส่ `return_exceptions=True` ไม่งั้น production จะเจอ task ค้างแบบเงียบ ๆ

สรุปความแตกต่าง: Blocking vs Async

ด้าน	Blocking	Async
เวลา 4 API calls (1s each)	4 วินาที	~1 วินาที
CPU ขณะรอ network	นอนรออยู่เฉยๆ	ทำงานอื่นระหว่างรอ
Concurrency model	ต้องใช้ thread/process	1 thread พอ
Memory overhead	สูง (thread ละ ~8MB)	ต่ำ (coroutine ละ ~KB)
เหมาะกับ	งาน CPU-heavy	งาน I/O-heavy (network, disk)

31.2 async def และ await — หัวใจของ Async

```
# =====
# กฎพื้นฐาน 3 ข้อ
# =====

# 1. function ที่มี "await" ข้างใน ต้องเป็น "async def"
async def fetch_ticker(symbol: str) -> dict:
    await asyncio.sleep(0.1)      # await บอกว่า "รอตรงนี้ได้ ไปทำอื่นก่อน"
    return {"symbol": symbol, "price": 65000.0, "volume": 1234.5}

# 2. "await" ใช้ได้แค่ภายใน "async def" เท่านั้น
async def main():
    ticker = await fetch_ticker("BTCUSDT")  # ถูกต้อง
    print(ticker)

# 3. เรียก async function จาก synchronous code ด้วย asyncio.run()
asyncio.run(main())

# =====
# coroutine คือ object ที่ยังไม่รัน — ต้อง await หรือ schedule
# =====

coro = fetch_ticker("BTCUSDT")      # สร้าง coroutine object
print(type(coro))                  # <class 'coroutine'> — ยังไม่รัน!
result = asyncio.run(coro)          # รัน coroutine
print(result)
```

► OUTPUT

```
{'symbol': 'BTCUSDT', 'price': 65000.0, 'volume': 1234.5}
<class 'coroutine'>
{'symbol': 'BTCUSDT', 'price': 65000.0, 'volume': 1234.5}
```

31.3 Event Loop — หัวใจของ asyncio

Event loop คือ "ผู้จัดการ" ที่วิ่งอยู่เบื้องหลัง คอยตรวจว่า coroutine ไหนพร้อมรันแล้ว และสลับการทำงานระหว่าง coroutines

Event Loop Lifecycle: _____

```

[Task A: fetch Binance]—await—>[รอ network] | | | | |
| | | | | ← สลับมา Task B | network มา | | | | | [Task B: fetch Bybit]—await—>[รอ network] | | | | |
| | | | | ← สลับมา Task C | | | | | [Task C: process signal] ← ไม่รอ network → รันได้ทันที |
| | | | | _____ ทั้งหมดรันใน thread เดียว — สลับกันที่จุด "await"

```

```
import asyncio

async def demonstrate_event_loop():
    """แสดงการสลับ context ระหว่าง coroutines"""

    async def task_a():
        print("Task A: เริ่มต้น")
        await asyncio.sleep(0.2)  # ยกมือออก event loop: "ไปทำอื่นก่อน"
        print("Task A: กลับมาแล้ว")
        return "A done"

    async def task_b():
        print("Task B: เริ่มต้น")
        await asyncio.sleep(0.2)  # ยกมือออก event loop: "ไปทำอื่นก่อน"
        print("Task B: กลับมาแล้ว")
        return "B done"
```

```

    print("Task B: เริ่มต้น")
    await asyncio.sleep(0.1)    # รอน้อยกว่า A → จบก่อน
    print("Task B: กลับมาแล้ว")
    return "B done"

async def task_c():
    print("Task C: ไม่รอ network เลย")
    # await asyncio.sleep(0) บอก event loop ให้สลับสักระยะ
    await asyncio.sleep(0)
    print("Task C: จบแล้ว")
    return "C done"

results = await asyncio.gather(task_a(), task_b(), task_c())
print(f"Results: {results}")

asyncio.run(demonstrate_event_loop())

```

```

► OUTPUT
Task A: เริ่มต้น
Task B: เริ่มต้น
Task C: ไม่รอ network เลย
Task C: จบแล้ว
Task B: กลับมาแล้ว
Task A: กลับมาแล้ว
Results: ['A done', 'B done', 'C done']

```

31.4 ทำไม Blocking Code ถึงอันตรายใน Async

อันตราย: ห้ามใช้ **Blocking call** ภายใน **async function**

```

# ❌ ผิด - time.sleep() บล็อก event loop ทั้งหมด!
async def bad_fetch(exchange: str) -> float:
    time.sleep(1)          # บล็อก! coroutine อื่นหยุดรอด้วย
    return 65000.0

# ❌ ผิด - requests.get() บล็อก event loop!
async def bad_rest_call(url: str):
    resp = requests.get(url)  # บล็อก! ใช้ aiohttp แทน
    return resp.json()

# ✓ ถูกต้อง - asyncio.sleep ไม่บล็อก
async def good_fetch(exchange: str) -> float:
    await asyncio.sleep(1)    # yield control กลับ event loop
    return 65000.0

# ✓ ถูกต้อง - ใช้ aiohttp สำหรับ HTTP
import aiohttp
async def good_rest_call(url: str):
    async with aiohttp.ClientSession() as session:
        async with session.get(url) as resp:
            return await resp.json()

```

Blocking (อย่าใช้ใน async)	Async alternative
<code>time.sleep(n)</code>	<code>await asyncio.sleep(n)</code>
<code>requests.get(url)</code>	<code>await session.get(url)</code> (aiohttp)
<code>open(file).read()</code>	<code>await aiofiles.open(file).read()</code>
<code>socket.recv()</code>	WebSocket library async
CPU-heavy loop	<code>await loop.run_in_executor(None, fn)</code>

ตัวอย่าง: Async price aggregator สำหรับ BTC pair

```
import asyncio
import random
from dataclasses import dataclass
from datetime import datetime

random.seed(7) # ให้ตัวเลขในหนังสือตรงกับที่คุณรันได้

@dataclass
class Ticker:
    exchange: str
    symbol: str
    price: float
    timestamp: datetime

async def fetch_ticker_mock(exchange: str, symbol: str,
                           base_price: float, delay: float) -> Ticker:
    """จำลอง REST API call"""
    await asyncio.sleep(delay)
    price = base_price + random.uniform(-50, 50)
    return Ticker(exchange, symbol, price, datetime.utcnow())

async def aggregate_btc_prices() -> dict[str, Ticker]:
    """ดึงราคา BTC จากหลาย exchange พร้อมกัน"""
    tickers = await asyncio.gather(
        fetch_ticker_mock("binance", "BTCUSDT", 65000.0, 0.08),
        fetch_ticker_mock("bybit", "BTCUSDT", 65005.0, 0.12),
        fetch_ticker_mock("okx", "BTCUSDT", 64995.0, 0.10),
    )
    result = {t.exchange: t for t in tickers}

    prices = [t.price for t in tickers]
    spread = max(prices) - min(prices)
    mid = sum(prices) / len(prices)
    print(f"Mid: ${mid:,.1f} Spread: ${spread:.1f}")
    return result

asyncio.run(aggregate_btc_prices())
```

► OUTPUT

Mid: \$64,987.5 Spread: \$60.0

🧠 gather คืนของมาเรียงตามลำดับไหน

```
tickers = await asyncio.gather(
    fetch_ticker_mock("binance", ..., 0.08), # เสร็จที่ 0.08s ← เร็วสุด
    fetch_ticker_mock("bybit", ..., 0.12),  # เสร็จที่ 0.12s ← ช้าสุด
    fetch_ticker_mock("okx", ..., 0.10),    # เสร็จที่ 0.10s
)
# tickers = [binance, bybit, okx] — เรียงตามที่เขียน ไม่ใช่ตามเวลาที่เสร็จ

result = {t.exchange: t for t in tickers} # dict comprehension
```

1. **gather** คืนผลเรียงตามลำดับที่ใส่เข้าไปเสมอ ไม่ใช่ตามลำดับที่ทำเสร็จ · okx เสร็จก่อน bybit แต่ยังอยู่หลังใน list · ข้อนี้ทำให้เขียนโค้ดง่ายขึ้นมาก เพราะจับคู่ผลลัพธ์กับ input ด้วยตำแหน่งได้เลย ไม่ต้องแปะ id · (ถ้าอยากได้ผลตามลำดับที่เสร็จจริง ต้องใช้ `asyncio.as_completed` แทน)
2. **เวลารวม = ตัวที่ช้าที่สุด ไม่ใช่ผลบวก** — 0.12 วินาที ไม่ใช่ $0.08+0.12+0.10 = 0.30$ · นี่คือทั้งหมดที่ async ให้ · ถ้าเพิ่ม exchange ที่ตอบใน 0.09s เข้าไปอีก 20 เจ้า เวลารวมก็ยัง 0.12 วินาทีเท่าเดิม
3. `{t.exchange: t for t in tickers}` — **dict comprehension** · อ่านว่า "สำหรับทุก t ใน tickers สร้างคู่ key: value" · ก่อนโคลนคือ key หลังโคลนคือ value · ที่นี้คือแปลง list เป็น dict ที่ค้นด้วยชื่อ exchange ได้ (`result["bybit"]`) แทนที่จะต้องจำว่าอยู่ตำแหน่งที่เท่าไร

🔴 [รอบสอง] "spread" ที่คำนวณได้ตรงนี้ ส่วนหนึ่งเป็นของปลอม

สามราคานี้ไม่ได้ถ่ายจากเวลาเดียวกัน — binance เป็นราคา ณ 0.08 วินาที · okx ณ 0.10 · bybit ณ 0.12 · การเอามาลบกันคือการเทียบราคาที่ห่างกันถึง **40 มิลลิวินาที**

ในตลาดที่ขยับเร็ว ส่วนต่างที่เห็นจึงปนกันสองอย่างแยกไม่ออก: **ส่วนต่างจริงระหว่างตลาด** (ของที่เทรดได้) กับ **ส่วนต่างที่เกิดจากถ่ายรูปคนละวินาที** (ของปลอมที่หายไปพอส่งคำสั่งจริง) · นี่คือสาเหตุอันดับหนึ่งที่ scanner หา arbitrage มือใหม่รายงานโอกาสเต็มจอ แต่พอกดจริงไม่มีสักอัน

ทางแก้ที่ระบบจริงใช้ — เรียงตามที่เราควรทำก่อนหลัง:

- **ติด timestamp ของ exchange มากับราคาทุกครั้ง** (ไม่ใช่เวลาที่เรารับได้) แล้วทิ้งคู่ที่ห่างกันเกินเกณฑ์ เช่น 50ms — สังเกตว่าโค้ดข้างบนเก็บ timestamp ไว้ใน Ticker แล้ว แต่ *ไม่เคยเอามาใช้เลย* · นั่นคือช่องที่ต้องเติม
- **ใช้ WebSocket แทน REST** — ได้ราคาไหลเข้ามาต่อเนื่อง จับคู่ตาม timestamp ได้แน่นกว่าการยิงถามเป็นครั้ง ๆ มาก (เป็นเหตุผลที่บทที่ 33 ทิ้งบ๊วด้วย WebSocket)
- **เทียบกับ latency ที่ต้องใช้จริง** — ถ้าส่วนต่างที่เจอเล็กกว่าที่เราขายได้ในเวลาที่เราส่งคำสั่งไปถึง มันไม่ใช่โอกาสไม่ว่าตัวเลขจะสวยแค่ไหน

อีกจุดที่ควรรู้: `datetime.utcnow()` คืนเวลาแบบ **naive** (ไม่มี timezone ติดมา) · ตั้งแต่ Python 3.12 ถูกประกาศเลิกใช้แล้ว และควรเปลี่ยนเป็น `datetime.now(timezone.utc)` · ในระบบเทรดเรื่องนี้ไม่ใช่แค่ความสวยงาม — เวลาแบบไม่มี timezone เอาไปลบกับเวลาที่มี timezone จะ **TypeError** ทันที และมักไปพังตอนต่อกับข้อมูลจาก exchange ที่ส่ง timezone มาด้วย

► **แบบฝึกหัด:** เพิ่ม timeout ให้ fetch — ถ้า exchange ไม่ตอบใน 0.5s ให้ข้ามไป

บทที่ 32: asyncio Patterns

แก่นของบทนี้

asyncio มี building blocks สำคัญ 4 อย่างสำหรับระบบ trading จริง: **gather()** สำหรับ concurrent fetch, **Queue** สำหรับ decouple producer/consumer, **wait_for()** สำหรับ timeout, และ **Task** สำหรับ background jobs ที่ cancel ได้

32.1 asyncio.gather() — รันพร้อมกัน

```
import asyncio
import random

random.seed(11)

async def fetch_ohlcv(exchange: str, symbol: str,
                     timeframe: str = "1m") -> list:
    """จำลองดึง OHLCV candle ล่าสุด"""
    await asyncio.sleep(0.1)
    p = 65000 + random.uniform(-200, 200)
    return [exchange, symbol, timeframe,
            {"open": p-10, "high": p+20, "low": p-30, "close": p,
             "volume": random.uniform(100, 500)}]

async def fetch_all_ohlcv():
    """ดึง OHLCV จากหลาย exchange/timeframe พร้อมกัน"""
    pairs = [
        ("binance", "BTCUSDT", "1m"),
        ("bybit", "BTCUSDT", "1m"),
        ("binance", "BTCUSDT", "5m"), # timeframe อื่นด้วย
        ("bybit", "BTCUSDT", "5m"),
    ]

    # gather ทุก coroutine พร้อมกัน
    results = await asyncio.gather(
        *[fetch_ohlcv(ex, sym, tf) for ex, sym, tf in pairs],
        return_exceptions=True # แทนที่จะ raise ทันที ให้เก็บ exception ใน result
    )

    for r in results:
        if isinstance(r, Exception):
            print(f"[ERROR] {r}")
        else:
            ex, sym, tf, candle = r
            print(f"{ex:8s} {sym} {tf:3s}: close={candle['close']:, .1f}")

asyncio.run(fetch_all_ohlcv())
```

► OUTPUT

```
binance BTCUSDT 1m : close=64,981.0
bybit   BTCUSDT 1m : close=65,169.7
binance BTCUSDT 5m : close=65,003.1
bybit   BTCUSDT 5m : close=64,873.9
```

return_exceptions=True — สวิตช์ที่เปลี่ยนพฤติกรรมทั้งระบบ

```
# ไม่ใส่ (ค่าเริ่มต้น): เจอ error ตัวแรก → เด้งออกทันที
results = await asyncio.gather(*tasks)
# → ถ้า bybit ล่ม เราไม่ได้ราคาของ binance กับ okx เลย ทั้งที่มันสำเร็จแล้ว

# ใส่: error กลายเป็น "ผลลัพธ์" ชนิดหนึ่ง วางเรียงในลิสต์ตามตำแหน่งเดิม
results = await asyncio.gather(*tasks, return_exceptions=True)
for r in results:
    if isinstance(r, Exception): # ← จึงต้องเช็คชนิดก่อนใช้เสมอ
        print(f"[ERROR] {r}")
```

1. **ค่าเริ่มต้นเหมาะกับ "ทั้งหมดหรือไม่เอาเลย"** — เช่นดึงขาสองข้างของ pair trade ถ้าขาดข้างหนึ่งก็คำนวณ spread ไม่ได้อยู่ดี · `return_exceptions=True` เหมาะกับ "ได้เท่าไหนเอาเท่านั้น" — เช่นสำรวจราคาจาก 10 ตลาด ล้มไป 2 ก็ยังทำงานต่อไป
 2. **ราคาที่ต่อจ่ายคือ `isinstance` ทุกครั้ง** — เพราะสมาชิกใน list ไม่ได้เป็นชนิดเดียวกันอีกต่อไป · *ลืมเช็ก = โปรแกรมจะพังที่บรรทัดถัดไปแทนด้วย error ที่ดูไม่ออกว่ามาจากไหน* (เช่น `TypeError: cannot unpack non-sequence`)
 3. **สิ่งที่มันไม่ทำ** — มันไม่ได้ทำให้ error หายไป และไม่ได้ `retry` ให้ · มันแค่ *เลื่อนการตัดสินใจ* ให้เราจัดการเอง · ถ้าเขียน `for` วนแล้วไม่เจอกรณี Exception เลย เท่ากับ **กลืน error เงียบ ๆ** ซึ่งอันตรายกว่าไม่ใส่ตั้งแต่แรก
- สังเกตว่าโค้ดข้างบนพิมพ์ `[ERROR]` ออกมาเฉย ๆ แล้วไปต่อ · ในระบบจริงตรงนี้ต้อง **ตัดสินใจ** ว่าราคาที่เราขาดไปทำให้สัญญาณใช้ไม่ได้หรือไม่ — การพิมพ์แล้วเดินหน้าต่อเหมือนไม่มีอะไรเกิดขึ้น คือวิธีที่ระบบเทรดขาดทุนแบบเงียบที่สุด

32.2 asyncio.Queue — Decouple Producer และ Consumer

ใน live trading มี 2 ส่วนที่ทำงานเร็วต่างกัน: **feed** รับข้อมูลเร็ว และ **signal engine** คำนวณช้ากว่า Queue เป็นตัวกลางที่ buffer ข้อมูลไว้

Producer (WebSocket feed) Consumer (Signal Engine) ————— tick มาทุก 10ms
—put→ asyncio.Queue —get→ คำนวณ signal ไม่ต้องรอ consumer (buffer ≤ 1000 items) ส่ง order

📌 **อ่านว่า:** โค้ดข้างล่างมี 3 ฟังก์ชัน แต่ **สัญญาณระหว่างกันมีแค่ 4 บรรทัด** — จบ 4 บรรทัดนี้ได้แล้วที่เลือกอ่านผ่านได้เลย: ฝั่งผลิตใช้ `await queue.put(x)` · ฝั่งบริโภคใช้ `tick = await queue.get()` · ฝั่งผลิตปิดท้ายด้วย `await queue.put(None)` เป็น **สัญญาณว่าหมดแล้ว** · ฝั่งบริโภคเจอ `None` เมื่อไหร่ก็ `break` · ที่เหลือคือการคำนวณ moving average ธรรมดาที่ไม่เกี่ยวกับ async เลย

```
import asyncio
import random
from dataclasses import dataclass, field
from datetime import datetime

@dataclass
class PriceTick:
    exchange: str
    symbol: str
    price: float
    timestamp: datetime = field(default_factory=datetime.utcnow)

async def price_producer(queue: asyncio.Queue, exchange: str,
                        symbol: str, n_ticks: int = 20):
    """จำลอง WebSocket feed ส่ง tick เข้า queue"""
    price = 65000.0
    for i in range(n_ticks):
        price += random.uniform(-50, 50)
        tick = PriceTick(exchange, symbol, round(price, 2))
        await queue.put(tick)
        print(f"[PROD] {exchange:8s} put tick #{i+1:2d}: {price:,.1f}")
        await asyncio.sleep(0.05) # จำลอง tick interval

    # ส่ง sentinel เพื่อบอก consumer ว่าจบแล้ว
    await queue.put(None)

async def signal_consumer(queue: asyncio.Queue, window: int = 5):
    """คำนวณ signal จาก tick ที่รับจาก queue"""
    prices = []
    processed = 0

    while True:
        tick = await queue.get() # รอจนกว่าจะมี item ใน queue
        if tick is None: # sentinel — จบ
            queue.task_done()
            break

        prices.append(tick.price)
        queue.task_done()
        processed += 1

    if len(prices) >= window:
        ma = sum(prices[-window:]) / window
        latest = prices[-1]
        signal = "LONG" if latest > ma else ("SHORT" if latest < ma else "HOLD")
        print(f"[CONS] tick #{processed:2d}: price={latest:,.1f} "
              f"signal={signal}")
```

```
f"MA(window)={ma:,.1f} - {signal}")

async def producer_consumer_demo():
    queue = asyncio.Queue(maxsize=100) # buffer สูงสุด 100 items

    # รัน producer และ consumer พร้อมกัน
    await asyncio.gather(
        price_producer(queue, "binance", "BTCUSDT", n_ticks=10),
        signal_consumer(queue, window=5),
    )
    print("จบ demo")

asyncio.run(producer_consumer_demo())
```

🧐 ทำไมต้องมี Queue — ทั้งที่เรียกฟังก์ชันตรง ๆ ก็ได้

```
# ถ้าไม่มี queue จะต้องเขียนแบบนี้ — ผลิตกับบริโภคมัดติดกัน
async def feed():
    while True:
        tick = await get_tick()
        compute_signal(tick) # - ถ้าตัวนี้ช้า tick ถัดไปก็ต้องรอ

# มี queue แล้ว: สองฝั่งวิ่งด้วยความเร็วของตัวเอง
await queue.put(tick) # ฝั่งผลิต - วางแล้วไปต่อทันที
tick = await queue.get() # ฝั่งบริโภค - ไม่มีของกินจนรอ ไม่กิน CPU
```

1. `await queue.get()` — **await** ตรงนี้แปลว่า "ถ้ายังไม่มีของ ก็นอนรอ แล้วปล่อยให้ตัวอื่นทำงานไปก่อน" · ต่างจาก `while queue.empty(): pass` ซึ่งจะเผา CPU 100% และแย่กว่านั้นคือ *ไม่ยอมปล่อยคิว* ทำให้ producer ไม่มีโอกาสวางของลงเลย → ค้างถาวร
2. `await queue.put(None)` — **sentinel** · เหตุที่ต้องมี เพราะ consumer เดาไม่ได้ว่า "queue ว่าง" แปลว่าจบแล้ว หรือแค่ producer ยังผลิตไม่ทัน · ทั้งสองสถานะหน้าตาเหมือนกันเป๊ะ · จึงต้องให้ producer **บอกออกมาตรง ๆ** ว่าหมดแล้ว · `None` ใช้ได้เพราะ tick จริงเป็น `PriceTick` ไม่มีทางเป็น `None`
3. `asyncio.Queue(maxsize=100)` — ถ้าของเต็ม 100 ชิ้น `await queue.put(...)` จะ **หยุดรอ** จนกว่าจะมีที่ว่าง · เรียกว่า **backpressure** — เป็นการบอก producer ว่า "ช้าลงหน่อย ตามไม่ทันแล้ว" · ถ้าใส่ `maxsize=0` (ไม่จำกัด) แล้ว consumer ตามไม่ทัน queue จะโดนกินแรมหมดเครื่อง
4. `asyncio.gather(producer, consumer)` — เริ่มทั้งคู่พร้อมกันในลูปเดียว · ไม่ใช้ thread ไม่มี lock ไม่มี race condition ให้กังวล เพราะทั้งคู่ **สลับกันที่จุด await เท่านั้น** ไม่มีทางถูกขัดจังหวะกลางคัน

⚠️ `task_done()` ในโค้ดนี้ไม่ได้ทำอะไรเลย — และควรรู้ว่าทำไม

`queue.task_done()` มีไว้คู่กับ `await queue.join()` เท่านั้น · `join()` แปลว่า "รอจนกว่าของทุกชิ้นที่เคย `put` จะถูกรายงานว่าทำเสร็จแล้ว" และมันนับจากจำนวนครั้งที่เรียก `task_done()`

ในบทนี้ **ไม่มีการเรียก `join()`** เลยสักที — เราใช้ sentinel `None` บอกจุดจบแทน · ดังนั้น `task_done()` ทุกบรรทัดในหนังสือเล่มนี้เป็น **พิธีกรรมล้วน ๆ** สอบอกก็ได้ผลเหมือนเดิม · ที่ยังเขียนไว้เพราะเป็นนิสัยที่ดีเวลาโค้ดขึ้นแล้วมีคนเดิม `join()` เข้ามาที่หลัง

🔗 **กฎ:** เห็น `task_done()` ที่ไหนให้มองหา `join()` ทันที · ถ้าไม่มีแปลว่าโค้ดนั้นใช้วิธีอื่นบอกจุดจบ (sentinel / cancel / event) — และถ้าไม่มีทั้งคู่ แปลว่าโปรแกรมเมอร์ไม่รู้ตัวตัวเองจบเมื่อไร

🔵 [รอบสอง] ในระบบเทรดจริง buffer ที่ใหญ่คือ **ปัญหา** ไม่ใช่ทางแก้

Queue เป็น **FIFO** — เข้าก่อนออกก่อน · นั่นถูกสำหรับงานทั่วไป แต่ **ผิดสำหรับราคา** · ถ้า signal engine ตามไม่ทันแล้วมี tick ค้างอยู่ 100 ชิ้น สิ่งที่มีนัยยะมาค่านวนคือ tick ที่ **เก่าที่สุด** · คุณจะส่งคำสั่งซื้อโดยอ้างอิงราคาเมื่อ 5 วินาทีที่แล้ว ทั้งที่ราคาล่าสุดวางอยู่ท้ายคิวแล้ว — และยิ่งตามไม่ทันนานเท่าไร ก็ยิ่งเทรดจากอดีตที่ไกลขึ้นเรื่อย ๆ

ทางเลือกที่ระบบจริงใช้ ขึ้นกับว่าข้อมูลนั้น **ทดแทนกันได้หรือไม่**:

- **ราคา / order book snapshot** — ของใหม่แทนที่ของเก่าได้สนิท · เก็บแค่ค่าล่าสุดพอ (ตัวแปรเดี่ยว หรือ `deque(maxlen=1)`) · ตกหล่นระหว่างทางไม่เป็นไร
- **fill / order update / trade execution** — **ห้ามทิ้งเด็ดขาด** ทุกเหตุการณ์มีความหมายทางบัญชี · ต้องใช้ queue ที่ไม่จำกัดขนาด และถ้ามันโตขึ้นเรื่อย ๆ นั่นคือ **สัญญาณเตือนให้หยุดระบบ** ไม่ใช่ให้ขยาย buffer

ดังนั้นคำถามที่ต้องตอบก่อนเลือก `maxsize` ไม่ใช่ "ควรตั้งเท่าไร" แต่คือ **"ถ้าตามไม่ทัน ควรทิ้งของเก่าหรือควรหยุดทั้งระบบ"** · ระบบจริงมักมีทั้งสองแบบวิ่งคู่กัน คนละ queue

32.3 asyncio.wait_for() — Timeout

```
import asyncio

async def place_order_mock(order_id: str, delay: float = 0.3) -> dict:
    """จำลอง REST API order placement"""
    await asyncio.sleep(delay)
    return {"order_id": order_id, "status": "FILLED", "avg_price": 65000.0}

async def place_order_with_timeout(order_id: str,
                                   timeout: float = 1.0) -> dict | None:
    """ส่ง order พร้อม timeout – ถ้าช้าเกินไปถือว่า fail"""
    try:
        result = await asyncio.wait_for(
            place_order_mock(order_id, delay=0.5),
            timeout=timeout
        )
        print(f"[OK] Order {order_id}: {result['status']}")
        return result

    except asyncio.TimeoutError:
        print(f"[WARN] Order {order_id}: timeout หลัง {timeout}s")
        # ต้อง query status ภายหลัง – order อาจ filled หรือ rejected
        return None

asyncio.run(place_order_with_timeout("ORD-001", timeout=1.0))
```

► OUTPUT

```
[OK] Order ORD-001: FILLED
```

32.4 asyncio.Task — Background Jobs และ Cancellation

```
import asyncio

async def heartbeat(interval: float = 1.0):
    """ส่ง heartbeat ทุก interval วินาที – background task"""
    count = 0
    while True:
        count += 1
        print(f"[HB] Heartbeat #{count}")
        await asyncio.sleep(interval)

async def main_trading_loop(duration: float = 3.5):
    """Main loop – รัน heartbeat เป็น background task"""

    # สร้าง Task – รันใน background ทันที
    hb_task = asyncio.create_task(heartbeat(1.0), name="heartbeat")

    print(f"[MAIN] เริ่ม trading loop ({duration}s)")
    await asyncio.sleep(duration)  # จำลองการเทรด

    # Cancel background task เมื่อจบ
    hb_task.cancel()
    try:
        await hb_task
    except asyncio.CancelledError:
        print("[MAIN] Heartbeat task ถูก cancel แล้ว")

    print("[MAIN] จบ")

asyncio.run(main_trading_loop())
```

► OUTPUT

```
[MAIN] เริ่ม trading loop (3.5s)
[HB] Heartbeat #1
[HB] Heartbeat #2
[HB] Heartbeat #3
[HB] Heartbeat #4
```

```
[MAIN] Heartbeat task ถูก cancel แล้ว
[MAIN] จบ
```

📖**อ่านว่า:** นับ heartbeat ให้ดี — ได้ 4 ครั้ง ไม่ใช่ 3 ทั้งที่รันแค่ 3.5 วินาที · เพราะใน heartbeat() คำสั่ง print อยู่ก่อน await asyncio.sleep() ครั้งแรกจึงยังทันที่ t=0 แล้วตามด้วย t=1, 2, 3 · ถ้าสลับให้ sleep มาก่อน จะได้ 3 ครั้ง (t=1,2,3) · ลำดับของ print กับ await เปลี่ยนจำนวนรอบที่ได้จริง — จุดที่พลาดกันบ่อยตอนนับ interval

32.5 asyncio.Semaphore — จำกัดจำนวน Concurrent Requests

ดึงข้อมูลย้อนหลังทีเดียว 500 วัน ถ้ายิงพร้อมกันหมด exchange จะแบนทันที — Semaphore คือตัวคุมว่า "ให้มีงานวิ่งอยู่พร้อมกันได้ไม่เกิน N ชิ้น"

📖**อ่านว่า:** คิดว่า Semaphore คือตะกร้าที่มีบัตรผ่านอยู่ N ใบ · ใครจะยิง request ต้องหยิบบัตรไปหนึ่งใบก่อน · บัตรหมดเมื่อไร คนที่มาทีหลังต้องยืนรอนจนกว่าจะมีคนคืนบัตร · async with sem: คือการหยิบบัตรตอนเข้าและคืนบัตรตอนออกโดยอัตโนมัติ — คืนให้แม้โค้ดข้างในจะ error กลางคัน ซึ่งเป็นเหตุผลทั้งหมดที่ใช้ async with แทนการเขียน acquire/release เอง (ลืมคืนบัตรครั้งเดียว = โปรแกรมค้างถาวร)

```
import asyncio

async def fetch_historical(symbol: str, date: str,
                           sem: asyncio.Semaphore) -> dict:
    """ดึงข้อมูล historical พร้อม rate limiting"""
    async with sem:
        # เข้า critical section — ไม่เกิน N พร้อมกัน
        await asyncio.sleep(0.1) # จำลอง API call
        return {"symbol": symbol, "date": date, "close": 65000.0}

async def bulk_fetch_historical(symbols: list[str],
                                dates: list[str]) -> list[dict]:
    """ดึงข้อมูลหลายวันพร้อมกัน แต่จำกัดไม่เกิน 5 request พร้อมกัน"""
    sem = asyncio.Semaphore(5) # อนุญาต 5 concurrent requests

    tasks = [
        fetch_historical(sym, date, sem)
        for sym in symbols
        for date in dates
    ]

    print(f"ดึงข้อมูล {len(tasks)} records ด้วย semaphore=5")
    results = await asyncio.gather(*tasks)
    return list(results)

asyncio.run(bulk_fetch_historical(
    symbols=["BTCUSD", "ETHUSD"],
    dates=["2024-01-01", "2024-01-02", "2024-01-03",
           "2024-01-04", "2024-01-05"]
))
```

► OUTPUT

ดึงข้อมูล 10 records ด้วย semaphore=5

🧑 2 บรรทัดที่ต้องอ่านให้ขาด

```
async with sem:
    await asyncio.sleep(0.1)

tasks = [
    fetch_historical(sym, date, sem)
    for sym in symbols
    for date in dates
]
```

1. `async with sem:` — เทียบเท่ากับ `await sem.acquire()` ตอนเข้า และ `sem.release()` ตอนออก · จุดสำคัญคือ **`release()` ถูกเรียกแม้โค้ดข้างในจะโยน exception** · ถ้าเขียนเองแล้ว API พังกลางทาง บัตรจะหายไปหนึ่งใบทุกครั้งที่ error — พอหายครบ N ใบ ระบบจะค้างแบบเจียบสนิทหาสาเหตุยากมาก
2. **list comprehension ที่มี for สองชั้น** — อ่านเรียงจากบนลงล่างเหมือน for ธรรมดาซ้อนกัน: `for sym` เป็นวงนอก `for date` เป็นวงใน · ได้ทุกคู่ที่จับกันได้ $2 \times 5 = 10$ · อย่าอ่านสลับ — ลำดับนี้กำหนดลำดับผลลัพธ์ที่ `gather` คืนกลับมา ซึ่งเรียงตามลำดับ task ที่ใส่เข้าไปเสมอ (ไม่ใช่ตามลำดับที่ทำเสร็จ)

3. สังเกตว่า `fetch_historical(...)` ในลิสต์ยัง**ไม่ได้รัน** — มันสร้าง coroutine object ทั้งไว้เฉย ๆ 10 ตัว · ทุกอย่างเริ่มพร้อมกันตอน `await asyncio.gather(*tasks)` บรรทัดเดียว · * คือการแกะลิสต์ออกเป็น argument ที่ละตัว เพราะ `gather` รับเป็น argument แยก ไม่ใช่ลิสต์

Semaphore ไม่ใช่ rate limiter — และความเข้าใจผิดนี้ทำให้โดนแบน

Semaphore คุ่ม "จำนวนงานที่วิ่งอยู่พร้อมกัน" ไม่ใช่ "จำนวน request ต่อวินาที" · สองอย่างนี้ไม่เท่ากัน และตัวแปลงระหว่างมันคือ *latency* ของ *exchange* ซึ่งเราควบคุมไม่ได้:

$$\text{req/s} \approx \text{จำนวนที่วิ่งพร้อมกัน} \div \text{latency}$$

ลองวัดจริงด้วยการจำลอง latency 0.1 วินาที:

```
import asyncio
import time

async def timed_bulk(n_requests: int, limit: int,
                    latency: float = 0.1) -> float:
    """อิง n_requests โดยให้วิ่งพร้อมกันได้ไม่เกิน limit - คืนเวลาที่ใช้"""
    sem = asyncio.Semaphore(limit)

    async def one_call():
        async with sem:
            await asyncio.sleep(latency) # จำลองเวลารอ exchange

    t0 = time.perf_counter()
    await asyncio.gather(*[one_call() for _ in range(n_requests)])
    return time.perf_counter() - t0

async def compare_limits():
    n, latency = 30, 0.1
    print(f"latency = {latency}s · ส่ง {n} requests")

    elapsed = {}
    for limit in (1, 5):
        elapsed[limit] = await timed_bulk(n, limit, latency)
        # วัดเป็น "กี่รอบของ latency" แทนวินาที - ไม่แกว่งตามภาระเครื่อง
        rounds = round(elapsed[limit] / latency)
        print(f" Semaphore({limit}) -> {rounds}>2d รอบของ latency")

    print(f" Semaphore(5) เร็วกว่า Semaphore(1) "
          f"{elapsed[1] / elapsed[5]:.0f} เท่า")

asyncio.run(compare_limits())
```

► OUTPUT

```
latency = 0.1s · ส่ง 30 requests
Semaphore(1) -> 30 รอบของ latency
Semaphore(5) -> 6 รอบของ latency
Semaphore(5) เร็วกว่า Semaphore(1) 5 เท่า
```

● อ่านตัวเลขนี้ให้ออก แล้วจะเห็นกับดักที่แท้จริง

คิดเป็น req/s เอง: Semaphore(5) ยิง 30 requests จบใน 6 รอบ $\times 0.1\text{s} = 0.6$ วินาที $\rightarrow$ **50 req/s** ไม่ใช่ 5 · แม้แต่ Semaphore(1) ที่ยิงทีละตัว เรียงกันก็ยิงได้ $30 \div 3.0 = 10 \text{ req/s}$ · ถ้า exchange จำกัดไว้ 10 req/s แปลว่า**ได้ข้างบนละเมิดขีดจำกัดตั้งแต่ Semaphore(2)**

ที่อันตรายกว่านั้นคือ**ทิศทางของความผิดพลาด** · เมื่อ exchange ตอบเร็วขึ้น (latency ลด) จำนวน req/s ของเรา**เพิ่มขึ้นเอง**โดยที่เราไม่ได้ทำอะไรเลย · แปลว่าโค้ดจะผ่านตอนทดสอบบนเน็ตช้า ๆ แล้วไปโดนแบนตอน deploy ขึ้น server ที่อยู่ใกล้ exchange — ซึ่งคือตอนที่แย่ที่สุดที่จะพัง

ของที่ต้องใช้จริงคือตัวคุมที่นับตามเวลา ไม่ใช่ตามจำนวนที่วิ่งพร้อมกัน — เช่น token bucket: เติมโควตา N ในทุก 1 วินาที ใครจะยังต้องมีโควตา · สังเกตว่ามันคุมอัตราโดยตรง จึงไม่ขึ้นกับ latency เลย

ในระบบจริงมักใช้**ทั้งคู่พร้อมกัน** คนละหน้าที่: token bucket กันโดนแบน · Semaphore กันไม่ให้เปิด connection พร้อมกันจนหน่วยความจำหรือ file descriptor หมด

► แบบฝึกหัด: สร้าง MultiQueue สำหรับรับข้อมูลจากหลาย exchange แล้วรวมเข้า Queue เดียว

บทที่ 33: WebSocket & Live Data

แก่นของบทนี้

WebSocket คือ connection แบบ "เปิดค้างไว้" ระหว่าง client และ server — ต่างจาก HTTP ที่ต้อง request ทุกครั้ง Exchange ส่ง tick/candle ให้เราทันทีที่มีการเปลี่ยนแปลง ทำให้ latency ต่ำกว่า polling มาก แต่ต้องจัดการ disconnect, reconnect, และ parse message เองทั้งหมด

33.1 Binance Futures WebSocket — รูปแบบ Message จริง

Binance Futures ส่ง kline (candle) stream ในรูปแบบ JSON นี้ เมื่อ subscribe ที่ `wss://fstream.binance.com/ws/btcusdt@kline_1m`

```
# ตัวอย่าง kline message จาก Binance Futures WebSocket (format จริง)
BINANCE_KLINE_MSG = {
  "e": "kline",           # event type
  "E": 1705123456789,     # event time (ms)
  "s": "BTCUSDT",         # symbol
  "k": {
    "t": 1705123440000,    # kline start time (ms)
    "T": 1705123499999,    # kline close time (ms)
    "s": "BTCUSDT",       # symbol
    "i": "1m",            # interval
    "f": 100,             # first trade ID
    "L": 200,             # last trade ID
    "o": "65000.10",       # open
    "c": "65123.45",       # close (current price)
    "h": "65200.00",       # high
    "l": "64950.00",       # low
    "v": "123.456",        # base asset volume (BTC)
    "n": 1250,            # number of trades
    "x": False,           # is this kline closed? False = ยังไม่ปิด candle
    "q": "8023456.78",     # quote asset volume (USDT)
    "V": "61.234",        # taker buy base volume
    "Q": "3987654.32",     # taker buy quote volume
  }
}

# ตัวอย่าง bookTicker message จาก Binance (best bid/ask)
BINANCE_BOOKTICKER_MSG = {
  "e": "bookTicker",
  "u": 400900217,         # order book updateId
  "s": "BTCUSDT",
  "b": "65000.00",        # best bid price
  "B": "5.000",          # best bid qty
  "a": "65001.00",        # best ask price
  "A": "3.500",          # best ask qty
  "T": 1705123456789,     # transaction time
  "E": 1705123456790,     # event time
}
```

33.2 Bybit WebSocket — รูปแบบ Message

```
# Bybit V5 WebSocket — wss://stream.bybit.com/v5/public/linear
# Subscribe message:
BYBIT_SUBSCRIBE = {
  "op": "subscribe",
  "args": ["kline.1.BTCUSDT"] # kline.{interval}.{symbol}
}

# Kline message จาก Bybit V5:
BYBIT_KLINE_MSG = {
  "topic": "kline.1.BTCUSDT",
  "data": [
    {

```

```

        "start": 1705123440000, # start time ms
        "end": 1705123499999, # end time ms
        "interval": "1", # interval in minutes
        "open": "65000.10",
        "close": "65123.45",
        "high": "65200.00",
        "low": "64950.00",
        "volume": "123.456", # base volume (BTC)
        "turnover": "8023456.78", # quote volume (USD)
        "confirm": False, # False = candle ยังไม่ปิด
        "timestamp": 1705123456789
    }
],
"ts": 1705123456789,
"type": "snapshot" # หรือ "delta"
}

```

33.3 WebSocket Client ที่ Reconnect อัตโนมัติ

 **โครงการคุณโค้ด — บล็อกถัดไปยาว 220 บรรทัด อ่านเป็น 3 ก้อน**

อย่าเพิ่งอ่านทีละบรรทัด · ก้อนใหญ่ก็คือของ 3 อย่างที่วางต่อกัน:

1. **Candle** (~10 บรรทัด) — กล่องเก็บข้อมูลแท่งเทียน 1 แท่ง ให้ทุกตลาดพูดภาษาเดียวกัน
2. **BinanceWebSocketClient** (~95 บรรทัด) — ตัวหลักที่ต้องเข้าใจจริง ๆ · ข้างในมี 5 เมธอด ทำหน้าที่ต่างกันชัดเจน (ดูตารางล่าง)
3. **BybitWebSocketClient** (~95 บรรทัด) — **โครงเหมือนข้อ 2 เกือบทั้งหมด** ต่างแค่ URL, รูปแบบข้อความ และวิธี subscribe · อ่านข้อ 2 ให้เข้าใจแล้วข้อ 3 จะกวาดตาผ่านได้

5 เมธอดใน client แต่ละตัว แบ่งหน้าที่แบบนี้:

เมธอด	หน้าที่	ระดับ
<code>__init__</code>	เก็บค่าตั้งต้น (symbol, interval, callback)	●
<code>stream_url</code>	ประกอบ URL ของ stream	●
<code>_parse_kline</code>	แปลง JSON ดิบของตลาด → Candle · จุดที่แต่ละตลาดต่างกันมากที่สุด	●
<code>_connect_and_listen</code>	ต่อ WebSocket แล้วรับข้อความ — หัวใจของ async	●
<code>run</code>	วนต่อใหม่เมื่อหลุด (reconnect + backoff) — หัวใจของความทนทาน	●

 **ถ้าเวลาน้อย:** อ่าน `_parse_kline` กับ `run` ให้เข้าใจก่อน — สองตัวนี้คือที่ที่ระบบจริงพึ่งบอยสุด

```

import asyncio
import json
import logging
import random
import websockets
from dataclasses import dataclass
from datetime import datetime
from typing import Callable, Optional

logger = logging.getLogger(__name__)

@dataclass
class Candle:
    exchange: str
    symbol: str
    interval: str
    open: float
    high: float
    low: float
    close: float
    volume: float
    is_closed: bool
    timestamp: datetime

```

```

class BinanceWebSocketClient:
    """
    Binance Futures WebSocket client
    - reconnect อัตโนมัติด้วย exponential backoff
    - parse kline messages
    - ส่ง Candle object เข้า callback
    """

    WS_URL = "wss://fstream.binance.com/ws"
    MAX_RECONNECT_DELAY = 60.0 # วินาที
    INITIAL_RECONNECT_DELAY = 1.0

    def __init__(self,
                  symbol: str,
                  interval: str,
                  on_candle: Callable[[Candle], None],
                  queue: Optional[asyncio.Queue] = None):
        self.symbol = symbol.lower()
        self.interval = interval
        self.on_candle = on_candle
        self.queue = queue
        self._stop_event = asyncio.Event()
        self._reconnect_delay = self.INITIAL_RECONNECT_DELAY

    @property
    def stream_url(self) -> str:
        return f"{self.WS_URL}/{self.symbol}@kline_{self.interval}"

    def _parse_kline(self, raw: dict) -> Optional[Candle]:
        """แปลง Binance kline message เป็น Candle object"""
        try:
            k = raw["k"]
            return Candle(
                exchange = "binance",
                symbol = raw["s"],
                interval = k["i"],
                open = float(k["o"]),
                high = float(k["h"]),
                low = float(k["l"]),
                close = float(k["c"]),
                volume = float(k["v"]),
                is_closed = bool(k["x"]),
                timestamp = datetime.utcfromtimestamp(raw["E"] / 1000)
            )
        except (KeyError, ValueError, TypeError) as e:
            logger.error(f"Binance parse error: {e} | raw={str(raw)[:200]}")
            return None

    async def _connect_and_listen(self):
        """เชื่อมต่อและฟัง messages - 1 connection attempt"""
        logger.info(f"Connecting to {self.stream_url}")
        async with websockets.connect(
            self.stream_url,
            ping_interval=20, # ส่ง ping ทุก 20s
            ping_timeout=10,
            close_timeout=5,
        ) as ws:
            logger.info(f"Connected: {self.symbol}@kline_{self.interval}")
            self._reconnect_delay = self.INITIAL_RECONNECT_DELAY # reset

            async for raw_msg in ws:
                if self._stop_event.is_set():
                    break

                try:
                    msg = json.loads(raw_msg)
                    if msg.get("e") == "kline":
                        candle = self._parse_kline(msg)
                        if candle is not None: # ข้ามถ้า parse ไม่ได้
                            self.on_candle(candle)

                    # ถ้ามี queue ให้ put ด้วย
                    if self.queue is not None:

```

```

        await self.queue.put(candle)

    except json.JSONDecodeError as e:
        logger.error(f"JSON decode error: {e}")
    except KeyError as e:
        logger.error(f"Unexpected message format: {e} | msg={raw_msg[:200]}")

async def run(self):
    """รัน WebSocket พร้อม exponential backoff reconnect"""
    while not self._stop_event.is_set():
        try:
            await self._connect_and_listen()

        except websockets.exceptions.ConnectionClosed as e:
            logger.warning(f"Connection closed: {e.code} {e.reason}")
        except websockets.exceptions.WebSocketException as e:
            logger.error(f"WebSocket error: {e}")
        except OSError as e:
            logger.error(f"Network error: {e}")

        if self._stop_event.is_set():
            break

        # Exponential backoff + jitter ป้องกัน thundering herd
        logger.info(f"Reconnecting in {self._reconnect_delay:.1f}s...")
        await asyncio.sleep(self._reconnect_delay)
        self._reconnect_delay = min(
            self._reconnect_delay * 2,
            self.MAX_RECONNECT_DELAY
        ) + random.uniform(0, 1.0)

    logger.info("WebSocket client stopped")

def stop(self):
    """หยุด client อย่างสะอาด"""
    self._stop_event.set()

class BybitWebSocketClient:
    """
    Bybit V5 WebSocket client
    endpoint: wss://stream.bybit.com/v5/public/linear
    """

    WS_URL = "wss://stream.bybit.com/v5/public/linear"
    MAX_RECONNECT_DELAY = 60.0
    INITIAL_RECONNECT_DELAY = 1.0

    def __init__(self,
                 symbol: str,
                 interval: str,
                 on_candle: Callable[[Candle], None],
                 queue: Optional[asyncio.Queue] = None):
        self.symbol = symbol.upper()
        self.interval = interval
        self.on_candle = on_candle
        self.queue = queue
        self._stop_event = asyncio.Event()
        self._reconnect_delay = self.INITIAL_RECONNECT_DELAY

    def _parse_kline(self, msg: dict) -> list[Candle]:
        """แปล Bybit kline message - อาจมีหลาย candle ใน 1 message"""
        candles = []
        for k in msg.get("data", []):
            try:
                candles.append(Candle(
                    exchange = "bybit",
                    symbol = self.symbol,
                    interval = self.interval,
                    open = float(k["open"]),
                    high = float(k["high"]),
                    low = float(k["low"]),
                    close = float(k["close"]),

```

```

        volume = float(k["volume"]),
        is_closed = bool(k.get("confirm", False)),
        timestamp = datetime.utcfromtimestamp(k["start"] / 1000)
    ))
except (KeyError, ValueError, TypeError) as e:
    logger.error(f"Bybit parse error: {e} | k={str(k)[:200]}")
return candles

async def _connect_and_listen(self):
    logger.info(f"Connecting to Bybit: {self.symbol} kline.{self.interval}")
    async with websockets.connect(
        self.WS_URL,
        ping_interval=20,
        ping_timeout=10,
    ) as ws:
        # ส่ง subscribe message
        sub_msg = json.dumps({
            "op": "subscribe",
            "args": [f"kline.{self.interval}.{self.symbol}"]
        })
        await ws.send(sub_msg)
        self._reconnect_delay = self.INITIAL_RECONNECT_DELAY

    async for raw_msg in ws:
        if self._stop_event.is_set():
            break
        try:
            msg = json.loads(raw_msg)
            if msg.get("op") == "subscribe":
                logger.info(f"Bybit subscribe: {msg.get('ret_msg')}")
                continue
            if "topic" in msg and "kline" in msg["topic"]:
                for candle in self._parse_kline(msg):
                    self.on_candle(candle)
                    if self.queue is not None:
                        await self.queue.put(candle)
        except (json.JSONDecodeError, KeyError) as e:
            logger.error(f"Parse error: {e}")

    async def run(self):
        while not self._stop_event.is_set():
            try:
                await self._connect_and_listen()
            except (websockets.exceptions.WebSocketException, OSError) as e:
                logger.warning(f"Connection error: {e}")
            if not self._stop_event.is_set():
                await asyncio.sleep(self._reconnect_delay)
                self._reconnect_delay = min(
                    self._reconnect_delay * 2, self.MAX_RECONNECT_DELAY
                ) + random.uniform(0, 1.0)

    def stop(self):
        self._stop_event.set()

```

👑 2 คำใหม่ใน `_connect_and_listen` — `async with` และ `async for`

```

async with websockets.connect(...) as ws:  # ①
    async for raw_msg in ws:               # ②
        ...

```

1. `async with ... as ws` — "เปิดการเชื่อมต่อใช้งานแล้วปิดให้อัตโนมัติเสมอไม่ว่าจะจบแบบปกติหรือ error". คำว่า `async` ข้างหน้าบอกว่าทั้ง `ขึ้นเปิดและขึ้นปิด` ต้องรอได้ (เพราะการต่อ/ตัด network ใช้เวลา). ถ้าไม่มี `with` แล้วเกิด error กลางทาง connection จะค้างเปิดทิ้งไว้
2. `async for raw_msg in ws` — "วนรับข้อความทีละอันเมื่อมันมาถึง". ต่างจาก `for` ปกติที่วนของที่มีครบอยู่แล้ว — ตัวนี้ **ไม่รู้ว่ามีกี่อัน และไม่รู้ว่าจะอันถัดไปจะมาเมื่อไหร่** ระหว่างที่รอมันปล่อยให้ `coroutine` อื่นทำงาน. ลูปนี้จะวนไปเรื่อย ๆ จนกว่าฝั่งตลาดจะปิดการเชื่อมต่อ

จำง่าย: `async with` = ยืมของแล้วคืนอัตโนมัติ. `async for` = รับของที่ทยอยมา

📖 **อ่านว่า:** บรรทัดที่ดูแปลกที่สุดใน `run()`: `min(delay * 2, MAX) + random.uniform(0, 1.0)` — อ่านว่า "รอนานขึ้น **เท่าตัว** ทุกครั้งที่ต่อไม่ติด แต่ไม่เกินเพดานแล้วบวกเศษสุ่มอีกนิด". ส่วน `* 2` คือ exponential backoff (อย่ากระหน่ำ server ที่กำลังมีปัญหา) ส่วน `random` คือ jitter — เหตุผลอยู่ใน

🔵 [รอบสอง] ทำไมต้องบวกเลขสุ่ม — thundering herd

ถ้า exchange ล่มแล้วมี bot 5,000 ตัวต่อไม่ติดพร้อมกัน ทุกตัวจะรอ 1s → 2s → 4s เป็นจังหวะเดียวกันเป๊ะ พอครบเวลา ทั้ง 5,000 ตัวก็กระโดดเข้าไปพร้อมกันอีกรอบ — server ที่เพิ่งจะฟื้นก็ล้มซ้ำทันที เรียกว่า **thundering herd** . การบวก `random.uniform(0, 1)` ทำให้แต่ละตัวตื่นไม่พร้อมกัน โหลดจึงกระจาย

อีกจุดที่ค้างใจออกแบบ: `run()` ดักเฉพาะ `ConnectionClosed`, `WebSocketException`, `OSError` — **ไม่ดัก Exception** เปลา่ ๆ เพราะถ้าดักหมด บั๊กในโค้ดคุณเอง (เช่น `_parse_kline` พัง) จะถูกกลืนแล้ววน reconnect ไปเรื่อย ๆ โดยไม่มีใครรู้ว่าระบบตายไปแล้ว . **ดักเฉพาะ error ที่คุณรู้วิธีจัดการ ที่เหลือปล่อยให้ดัง** — นี่คือกฎที่แยกโค้ด production ออกจากโค้ด tutorial

สังเกต `self._reconnect_delay = INITIAL` ที่ถูก reset *หลังต่อติดแล้ว* — ถ้าลืมบรรทัดนี้ ระบบที่หลุดบ่อย ๆ จะค่อย ๆ ถ่างเป็นรอบ 60 วินาทีถาวร ทั้งที่ต่อติดได้ทันทีมานานแล้ว

33.4 Dual-Feed WebSocket — รับ Binance + Bybit พร้อมกัน

Running Example: รับ kline จาก Binance และ Bybit พร้อมกัน → ส่งเข้า Queue เดียว

นี่คือ core ของ live pair strategy — เราต้องการราคาจากทั้ง 2 exchange ในเวลาเดียวกันเพื่อคำนวณ spread

```
import asyncio
import json
import logging
from collections import deque
from typing import Optional

logger = logging.getLogger(__name__)

async def run_dual_feed(symbol: str = "BTCUSDT",
                        interval: str = "1",
                        max_candles: int = 100,
                        run_seconds: float = 30.0):
    """
    รับ Binance + Bybit WebSocket พร้อมกัน
    ส่ง candle ทั้งสองเข้า shared queue
    หยุดเองหลัง run_seconds วินาที (สำหรับ demo)
    """
    shared_queue: asyncio.Queue[Candle] = asyncio.Queue(maxsize=500)
    stop_event = asyncio.Event()

    # Buffer สำหรับคำนวณ spread
    candle_buf: dict[str, deque] = {
        "binance": deque(maxlen=max_candles),
        "bybit": deque(maxlen=max_candles),
    }

    def on_candle(candle: Candle):
        # Callback รับ synchronously — ใช้แค่ log
        if candle.is_closed:
            logger.debug(f"[{candle.exchange}] Closed candle "
                        f"{candle.symbol} close={candle.close}")

    # สร้าง clients
    binance_client = BinanceWebSocketClient(
        symbol=symbol, interval=f"{interval}m",
        on_candle=on_candle, queue=shared_queue
    )
    bybit_client = BybitWebSocketClient(
        symbol=symbol, interval=interval,
        on_candle=on_candle, queue=shared_queue
    )

    async def process_candles():
        """Consumer: คำนวณ spread เมื่อได้ candle ปิดจากทั้งสอง"""
        while not stop_event.is_set():
```

```

try:
    candle = await asyncio.wait_for(
        shared_queue.get(), timeout=1.0
    )
except asyncio.TimeoutError:
    continue

if candle.is_closed:
    candle_buf[candle.exchange].append(candle.close)

# คำนวณ spread เมื่อมีข้อมูลทั้งสอง exchange
if (len(candle_buf["binance"]) > 0 and
    len(candle_buf["bybit"]) > 0):
    b_price = candle_buf["binance"][-1]
    y_price = candle_buf["bybit"][-1]
    spread = y_price - b_price
    logger.info(f"Spread: {spread:+.2f} "
                f"(Binance={b_price:.1f}, Bybit={y_price:.1f})")

shared_queue.task_done()

async def stopper():
    await asyncio.sleep(run_seconds)
    stop_event.set()
    binance_client.stop()
    bybit_client.stop()
    logger.info("Stop event set")

# รันทุก task พร้อมกัน
await asyncio.gather(
    binance_client.run(),
    bybit_client.run(),
    process_candles(),
    stopper(),
    return_exceptions=True
)

# ใช้งาน:
# asyncio.run(run_dual_feed("BTCUSDT", "1", run_seconds=300))

```

ข้อควรระวัง: WebSocket ใน Production

- **ตรวจสอบ timestamp** — ถ้า message เก่ากว่า 5 วินาที อาจเป็น stale data → อย่าเทรด
- **Bybit ต้องการ re-subscribe** หลัง reconnect — ต้องส่ง subscribe message ใหม่ทุกครั้ง
- **Binance จำกัด 10 connections** ต่อ IP — ระวังถ้ารัน multiple instances
- **ใช้ testnet ก่อน**: Binance testnet: `wss://stream.binancefuture.com/ws` , Bybit testnet: `wss://stream-testnet.bybit.com/v5/public/linear`

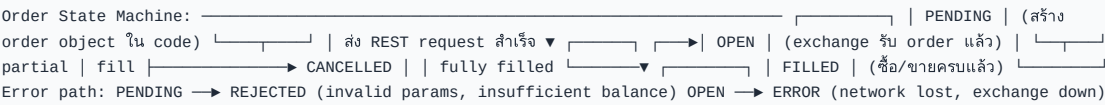
► **แบบฝึกหัด**: เพิ่ม staleness check — ถ้า candle เก่ากว่า 30 วินาที ให้ log warning

บทที่ 34: Order Execution

แก่นของบทนี้

การส่ง order จริงซับซ้อนกว่า backtest มาก: ต้องมี HMAC signature, จัดการ rate limit, retry เมื่อ network error (แต่ไม่ retry ถ้า order ถูก reject), และติดตาม state ของ order ตั้งแต่ PENDING จนถึง FILLED หรือ CANCELLED

34.1 Order State Machine



```
from enum import Enum, auto
from dataclasses import dataclass, field
from typing import Optional
import time

class OrderStatus(Enum):
    PENDING = auto() # สร้างใน code แต่ยังไม่ส่ง
    OPEN = auto() # ส่งไป exchange แล้ว รอ fill
    PARTIALLY_FILLED = auto()
    FILLED = auto() # fill ครบแล้ว
    CANCELLED = auto()
    REJECTED = auto() # exchange ปฏิเสธ
    ERROR = auto() # network/unknown error

class OrderSide(Enum):
    BUY = "Buy"
    SELL = "Sell"

class OrderType(Enum):
    MARKET = "MARKET"
    LIMIT = "LIMIT"

@dataclass
class Order:
    client_order_id: str # ID ที่เราสร้างเอง (UUID หรือ timestamp)
    symbol: str
    side: OrderSide
    order_type: OrderType
    qty: float
    price: Optional[float] = None # None สำหรับ market order

    # Fields ที่ exchange จะส่งกลับมา
    exchange_order_id: Optional[str] = None
    status: OrderStatus = OrderStatus.PENDING
    filled_qty: float = 0.0
    avg_price: float = 0.0
    created_at: float = field(default_factory=time.time)
    updated_at: Optional[float] = None
    reject_reason: Optional[str] = None

    @property
    def is_terminal(self) -> bool:
        """คืน True ถ้า order จบแล้ว (ไม่สามารถเปลี่ยนได้อีก)"""
        return self.status in (
            OrderStatus.FILLED,
            OrderStatus.CANCELLED,
            OrderStatus.REJECTED,
            OrderStatus.ERROR,
        )

    @property
```



```
def remaining_qty(self) -> float:
    return self.qty - self.filled_qty

def __repr__(self):
    return (f"Order({self.client_order_id} "
            f"{self.side.value} {self.qty} {self.symbol} "
            f"@ {'MKT' if self.price is None else self.price} "
            f"[{self.status.name}])")
```

34.2 REST API Wrapper พร้อม Rate Limiting และ Retry

📌 โครงก่อนคู่มือ — บล็อกถัดไปยาว ~240 บรรทัด อ่านเป็น 3 ชั้น

อย่าอ่านไล่บรรทัด · คลาสนี้มีชั้นชัดเจน แต่ละชั้นเรียกชั้นล่างลงไป:

ชั้น	เมธอด	หน้าที่	ระดับ
① เปลี่ยนออก สิ่งที่โค้ดเราเรียกใช้	place_market_order	ส่งคำสั่งซื้อขาย — แปลง argument เป็น payload ของตลาด	●
	place_limit_order		
	cancel_order	ยกเลิก / เช็คสถานะ	●
	get_order_status		
② แกนกลาง ทุกคำสั่งวิ่งผ่านตรงนี้	_request	หัวใจของทั้งคลาส — rate limit · retry · timeout · แปล error ~60 บรรทัด และเป็นวิธีที่ระบบจริงพึ่งพามากที่สุด	●
③ งานประกอบ	_sign	เซ็นลายเซ็น HMAC ให้ตลาดยืนยันตัวตน	●
	__aenter__ __aexit__	เปิด/ปิด session อัตโนมัติ — ทำให้ใช้กับ async with ได้ (หลักการเดียวกับ with ในบทที่ 8.5)	●

⚠️ ถ้าเวลาน้อย: อ่าน _request อย่างเดียวก็น่าจะพอแล้ว — เมธอดข้างบนทั้งหมดเป็นแค่เปลือกที่เรียกมัน · และ ชิดเส้นได้ลำดับใน _request :
จำกัดอัตรา — เซ็น → ยิง → ถ้าพลาดค่อยลองใหม่ ลำดับนี้สำคัญกว่ารายละเอียดของแต่ละชั้น

```
import asyncio
import hashlib
import hmac
import json
import os
import time
import logging
from urllib.parse import urlencode
from typing import Optional

import aiohttp

logger = logging.getLogger(__name__)

class BybitOrderClient:
    """
    Bybit V5 REST API wrapper
    - Rate limiting ด้วย asyncio.Semaphore
    - HMAC-SHA256 signature
    - Retry with exponential backoff สำหรับ network error
    - ไม่ retry ถ้า exchange reject (4xx)
    """

    # Testnet: https://api-testnet.bybit.com
    BASE_URL = "https://api.bybit.com"
    RECV_WINDOW = 5000  # ms

    def __init__(self,
                 api_key: Optional[str] = None,
                 api_secret: Optional[str] = None,
                 testnet: bool = True,
                 max_concurrent: int = 5):
        # ✓ โหลดจาก env var — ห้าม hardcode!
        self.api_key = api_key or os.environ.get("BYBIT_API_KEY", "")
```

```

self.api_secret = api_secret or os.environ.get("BYBIT_API_SECRET", "")

if testnet:
    self.BASE_URL = "https://api-testnet.bybit.com"
    logger.warning("🚧 TESTNET - 🚧 real money")

self._sem = asyncio.Semaphore(max_concurrent)
self._session: Optional[aiohttp.ClientSession] = None

async def __aenter__(self):
    self._session = aiohttp.ClientSession()
    return self

async def __aexit__(self, *args):
    if self._session:
        await self._session.close()

def _sign(self, params: dict) -> str:
    """สร้าง HMAC-SHA256 signature"""
    timestamp = str(int(time.time() * 1000))
    params_str = urlencode(sorted(params.items()))
    payload = f"{timestamp}{self.api_key}{self.RECV_WINDOW}{params_str}"
    signature = hmac.new(
        self.api_secret.encode("utf-8"),
        payload.encode("utf-8"),
        hashlib.sha256
    ).hexdigest()
    return signature, timestamp

async def _request(self, method: str, path: str,
                  params: dict = None,
                  max_retries: int = 3) -> dict:
    """ส่ง signed request พร้อม retry"""
    params = params or {}
    signature, timestamp = self._sign(params)
    headers = {
        "X-BAPI-API-KEY": self.api_key,
        "X-BAPI-SIGN": signature,
        "X-BAPI-TIMESTAMP": timestamp,
        "X-BAPI-RECV-WINDOW": str(self.RECV_WINDOW),
        "Content-Type": "application/json",
    }
    url = f"{self.BASE_URL}{path}"

    for attempt in range(max_retries):
        async with self._sem: # rate limiting
            try:
                if method == "GET":
                    async with self._session.get(
                        url, params=params, headers=headers,
                        timeout=aiohttp.ClientTimeout(total=10)
                    ) as resp:
                        data = await resp.json()
                else:
                    async with self._session.post(
                        url, json=params, headers=headers,
                        timeout=aiohttp.ClientTimeout(total=10)
                    ) as resp:
                        data = await resp.json()

                # Bybit ret_code: 0 = OK, อื่นๆ = error
                if data.get("retCode") != 0:
                    ret_msg = data.get("retMsg", "unknown")
                    ret_code = data.get("retCode")

                    # Error ที่ไม่ควร retry
                    NO_RETRY_CODES = {
                        10001, # Parameter error
                        10004, # Sign check failed
                        110007, # Insufficient balance
                        110013, # Position is in liquidation
                        110025, # Position not exist
                    }

```

```

        if ret_code in NO_RETRY_CODES:
            raise ValueError(
                f"Exchange rejected [{ret_code}]: {ret_msg}"
            )

        logger.warning(f"API error [{ret_code}]: {ret_msg}, "
                       f"attempt {attempt+1}/{max_retries}")
        if attempt == max_retries - 1:
            raise RuntimeError(
                f"Exchange error หลังลอง {max_retries} ครั้ง "
                f"[{ret_code}]: {ret_msg}"
            )
        # ⚠ ต้อง continue - ถ้าปล่อยให้ไหลลงไป return
        # error ที่ควรลองใหม่จะถูกคืนออกไปเลย ไม่เคย retry
        await asyncio.sleep(2 ** attempt)
        continue

    # มาถึงตรงนี้ = retCode == 0 = สำเร็จจริง
    return data

except (aiohttp.ClientError, asyncio.TimeoutError) as e:
    if attempt == max_retries - 1:
        raise
    delay = 2 ** attempt
    logger.warning(f"Network error: {e}, retry in {delay}s")
    await asyncio.sleep(delay)

raise RuntimeError(f"Failed after {max_retries} attempts")

async def place_market_order(self,
                              symbol: str,
                              side: OrderSide,
                              qty: float,
                              client_order_id: str) -> Order:
    """ส่ง market order"""
    order = Order(
        client_order_id=client_order_id,
        symbol=symbol,
        side=side,
        order_type=OrderType.MARKET,
        qty=qty,
    )

    try:
        resp = await self._request("POST", "/v5/order/create", {
            "category": "linear",
            "symbol": symbol,
            "side": side.value,
            "orderType": "Market",
            "qty": str(qty),
            "orderLinkId": client_order_id,
        })

        if resp.get("retCode") == 0:
            result = resp["result"]
            order.exchange_order_id = result.get("orderId")
            order.status = OrderStatus.OPEN
            logger.info(f"Order placed: {order}")
        else:
            order.status = OrderStatus.REJECTED
            order.reject_reason = resp.get("retMsg")
            logger.error(f"Order rejected: {order.reject_reason}")

    except ValueError as e:
        order.status = OrderStatus.REJECTED
        order.reject_reason = str(e)
        logger.error(f"Order rejected by exchange: {e}")

    except Exception as e:
        order.status = OrderStatus.ERROR
        order.reject_reason = str(e)
        logger.error(f"Order error: {e}")

```

```

        order.updated_at = time.time()
        return order

    async def place_limit_order(self,
                                symbol: str,
                                side: OrderSide,
                                qty: float,
                                price: float,
                                client_order_id: str) -> Order:
        """สั่ง limit order"""
        order = Order(
            client_order_id=client_order_id,
            symbol=symbol,
            side=side,
            order_type=OrderType.LIMIT,
            qty=qty,
            price=price,
        )

        try:
            resp = await self._request("POST", "/v5/order/create", {
                "category": "linear",
                "symbol": symbol,
                "side": side.value,
                "orderType": "Limit",
                "qty": str(qty),
                "price": str(price),
                "timeInForce": "PostOnly", # maker order เท่านั้น
                "orderLinkId": client_order_id,
            })

            if resp.get("retCode") == 0:
                result = resp["result"]
                order.exchange_order_id = result.get("orderId")
                order.status = OrderStatus.OPEN
                logger.info(f"Limit order placed: {order}")
            else:
                order.status = OrderStatus.REJECTED
                order.reject_reason = resp.get("retMsg")

        except ValueError as e:
            order.status = OrderStatus.REJECTED
            order.reject_reason = str(e)

        except Exception as e:
            order.status = OrderStatus.ERROR
            order.reject_reason = str(e)

        order.updated_at = time.time()
        return order

    async def cancel_order(self,
                           symbol: str,
                           client_order_id: str) -> bool:
        """ยกเลิก order"""
        resp = await self._request("POST", "/v5/order/cancel", {
            "category": "linear",
            "symbol": symbol,
            "orderLinkId": client_order_id,
        })

        success = resp.get("retCode") == 0
        if success:
            logger.info(f"Cancelled order {client_order_id}")
        else:
            logger.warning(f"Cancel failed: {resp.get('retMsg')}")
        return success

    async def get_order_status(self, symbol: str,
                               client_order_id: str) -> Optional[Order]:
        """ดึง order status จาก exchange"""
        resp = await self._request("GET", "/v5/order/realtime", {
            "category": "linear",
            "symbol": symbol,

```

```

        "orderLinkId": client_order_id,
    })
    if resp.get("retCode") != 0:
        return None

    items = resp["result"].get("list", [])
    if not items:
        return None

    item = items[0]
    STATUS_MAP = {
        "New": OrderStatus.OPEN,
        "PartiallyFilled": OrderStatus.PARTIALLY_FILLED,
        "Filled": OrderStatus.FILLED,
        "Cancelled": OrderStatus.CANCELLED,
        "Rejected": OrderStatus.REJECTED,
    }

    order = Order(
        client_order_id = client_order_id,
        symbol = symbol,
        side = OrderSide.BUY if item["side"] == "Buy" else OrderSide.SELL,
        order_type = OrderType.MARKET if item["orderType"] == "Market" else OrderType.LIMIT,
        qty = float(item["qty"]),
        price = float(item["price"]) if item.get("price") else None,
        exchange_order_id = item["orderId"],
        status = STATUS_MAP.get(item["orderStatus"], OrderStatus.ERROR),
        filled_qty = float(item.get("cumExecQty", 0)),
        avg_price = float(item.get("avgPrice", 0)),
    )
    return order

```

🔗 และ `request` — ลำดับ 4 ขั้นที่ทุกคำสั่งต้องผ่าน

```

for attempt in range(max_retries):          # ④ วงลองใหม่
    async with self._sem:                   # ① จำกัดอัตรา
        try:
            async with self._session.post(...) as resp:  # ③ ยิง
                data = await resp.json()
                if data.get("retCode") != 0:  # ตลาดตอบว่าไม่สำเร็จ
                    ...
                    continue                # ← ลองใหม่
                return data                  # สำเร็จจริงเท่านั้น
        except (ClientError, TimeoutError):
            await asyncio.sleep(2 ** attempt)  # ④ กอยแบบทวีคูณ

```

1. `async with self._sem` — `_sem` คือ `asyncio.Semaphore` = "บัตรคิว" มีบัตรจำกัด ใครไม่ได้บัตรต้องรอ. **นี่คือสิ่งที่กันเราจากการยิงเกิน `ratelimit` แล้วโดนแบน** — ไม่ใช่การ `sleep` เดาสุ่ม
2. `self._sign(params)` — ตลาดต้องรู้ว่าเป็นเราจริง จึงต้องแนบลายเซ็น HMAC ที่คำนวณจาก `secret key + timestamp · timestamp` ทำให้ `request` เก่าใช้ซ้ำไม่ได้
3. **ยิงแล้วอ่านคำตอบ** — สังเกตว่ามีการตรวจ **สองขั้น**: ขั้นแรกคือ `network` สำเร็จไหม (จับด้วย `except`) ขั้นที่สองคือ **ตลาดตอบว่าสำเร็จไหม** (`retCode != 0`) · **HTTP 200 ไม่ได้แปลว่าออเดอร์ผ่าน** — นี่คือจุดที่คนพลาดมากที่สุด
4. `2 ** attempt` — รอ 1 → 2 → 4 วินาที (exponential backoff เหมือนบทที่ 33) เพราะถ้าตลาดกำลังมีปัญหา การยิงถี่ ๆ ยิ่งทำให้แย่ลง

จำโครงสร้าง: `เข้าคิว` → `เซ็นชื่อ` → `ยิง` → `ถ้าพลาดถอยแล้วลองใหม่`: ส่วน `place_market_order` / `cancel_order` ทั้งหมดเป็นแค่เปลือกที่เตรียม `params` แล้วเรียกเมธอดนี้

❌ กับดักที่เคยอยู่ในโค้ดนี้จริง — `retry` ที่ไม่เคย `retry`

เดิมนิพจน์ `return data` ถูกวางไว้ระดับเดียวกับ `if data.get("retCode") != 0`: ผลคือเมื่อเจอ `error` ที่ควรลองใหม่โค้ดจะ `logger.warning("... attempt 1/3")` แล้วคืนค่าที่ล้มเหลวออกไปทันที — ลูป `for attempt in range(3)` ไม่เคยวนรอบที่สองเลย

อันตรายเป็นพิเศษเพราะ `log` จะบอกว่า `"attempt 1/3"` ทำให้ดูเหมือนระบบกำลัง `retry` อยู่ และฟังก์ชันคืนค่าปกติไม่มี `exception` · ผู้เรียกได้ `data` ที่มี `retCode != 0` ไปใช้ต่อโดยไม่รู้ตัว

🚫 **วิธีจับบั๊กประเภทนี้:** ในลูป retry ให้ใส่ดูว่าทุกเส้นทางที่ไม่สำเร็จ จบด้วย `continue` หรือ `raise` - ถ้ามีเส้นทางไหน "ไหลลง" ไปถึง `return` ได้ทั้งที่ยังไม่สำเร็จ นั่นคือ retry ที่ตายแล้ว

🔵 [รอบสอง] จุดที่โค้ดนี้ยังไม่สมบูรณ์ (ตั้งใจให้เห็น)

- **ถือ semaphore ระหว่าง sleep** — `await asyncio.sleep(delay)` อยู่ข้างใน `async with self._sem` แปลว่าระหว่างรอ backoff เรายังกินบัตรคิวอยู่ ทำให้ request อื่นที่พร้อมจะต้องรอไปด้วย - ในระบบจริงควรออกจาก semaphore ก่อนแล้วค่อย sleep
- **ไม่มี idempotency ตอน timeout** — ถ้า network timeout ระหว่างส่งออเดอร์ เราไม่รู้ว่าตลาดได้รับหรือยัง - การ retry อาจทำให้เปิดสองไม้ - ทางแก้จริงคือส่ง `orderId` (client order id) ที่ไม่ซ้ำ แล้วให้ตลาดปฏิเสธของซ้ำเอง — โค้ดนี้มี `client_order_id` อยู่แล้วแต่ยังไม่ได้ใช้เพื่อการนี้
- **NO_RETRY_CODES เป็นรายการที่ต้องดูแล** — โค้ดที่ไม่อยู่ในลิสต์จะถูก retry หมด - ถ้าตลาดเพิ่ม error code ใหม่ที่ retry ไม่ได้ (เช่นบัญชีถูกระงับ) ระบบจะยิงซ้ำไปเรื่อย ๆ - ระบบ production ควร **default เป็นไม่ retry** แล้วระบุเฉพาะโค้ดที่ retry ได้แทน — ปลอดภัยกว่าเมื่อเจอของที่ไม่รู้จัก

34.3 ทดสอบบน Testnet

```
import asyncio
import uuid

async def demo_order_flow():
    """
    ทดสอบ order flow บน Bybit Testnet
    ต้องตั้ง env var:
        export BYBIT_API_KEY=your_testnet_key
        export BYBIT_API_SECRET=your_testnet_secret
    """
    async with BybitOrderClient(testnet=True) as client:

        # สร้าง unique client order ID
        coid = f"PAIR-{uuid.uuid4().hex[:8].upper()}"

        print(f"Placing market BUY 0.001 BTCUSD (coid={coid})")
        order = await client.place_market_order(
            symbol = "BTCUSD",
            side = OrderSide.BUY,
            qty = 0.001,
            client_order_id = coid,
        )
        print(f"Result: {order}")

        # รอให้ order settle
        await asyncio.sleep(1.0)

        # ดึง status
        updated = await client.get_order_status("BTCUSD", coid)
        if updated:
            print(f"Updated status: {updated.status.name}, "
                  f"filled={updated.filled_qty}, avg={updated.avg_price}")

# asyncio.run(demo_order_flow())
```

ข้อควรระวัง: Order Execution ในโลกจริง

- **ห้าม retry ทุก error** — ถ้า network timeout แล้ว retry → order อาจถูกส่ง 2 ครั้ง ใช้ `client_order_id` เพื่อ idempotency
- **Market order slip ได้** — ราคา fill อาจต่างจากราคาตลาดมาก โดยเฉพาะ pair ที่ volume น้อย
- **ตรวจ balance ก่อนส่ง order** — error 110007 (insufficient balance) ไม่ควรเกิดถ้าระบบออกแบบดี
- **Log ทุก order response** รวมถึง timestamp, `exchange_order_id`, และ status

► แบบฝึกหัด: เขียน `OrderTracker` class ที่เก็บ history ของทุก order และ query ได้

บทที่ 35: Live System Integration

แก่นของบทนี้

ประกอบทุกอย่างเข้าด้วยกันเป็น live trading system ที่ production-ready: logging ที่ structured, health check loop, kill switch ที่ทำงานได้จริง, graceful shutdown, และ pre-flight checklist ก่อน go-live

35.1 Structured Logging

ใน live system logging ต้องเป็น structured (JSON) เพื่อ search และ filter ง่าย — ไม่ใช่แค่ print string

```
import logging
import json
import sys
from datetime import datetime, timezone

class StructuredFormatter(logging.Formatter):
    """
    Formatter ที่ output JSON — ง่ายต่อการ search ด้วย jq หรือ ELK stack
    """

    def format(self, record: logging.LogRecord) -> str:
        log_obj = {
            "ts": datetime.now(timezone.utc).isoformat(),
            "level": record.levelname,
            "logger": record.name,
            "msg": record.getMessage(),
        }

        # เพิ่ม extra fields ถ้ามี
        for key in ("symbol", "order_id", "price", "side", "pnl", "error"):
            if hasattr(record, key):
                log_obj[key] = getattr(record, key)

        if record.exc_info:
            log_obj["exception"] = self.formatException(record.exc_info)

        return json.dumps(log_obj, ensure_ascii=False)

def setup_logging(level: str = "INFO",
                  log_file: str = None) -> logging.Logger:
    """ตั้งค่า logging สำหรับ live system"""
    logger = logging.getLogger("live_trading")
    logger.setLevel(getattr(logging, level.upper()))

    # Console handler
    console = logging.StreamHandler(sys.stdout)
    console.setFormatter(StructuredFormatter())
    logger.addHandler(console)

    # File handler (ถ้าระบุ)
    if log_file:
        fh = logging.FileHandler(log_file, encoding="utf-8")
        fh.setFormatter(StructuredFormatter())
        logger.addHandler(fh)

    return logger

# ใช้งาน
logger = setup_logging(level="INFO", log_file="live_trading.log")

# Log พร้อม extra context
logger.info("Order placed", extra={
    "symbol": "BTCUSD",
    "order_id": "ORD-001",
    "side": "BUY",
```

```
"price": 65000.0
})
```

► OUTPUT

```
{"ts": "2024-01-15T10:30:00.123456+00:00", "level": "INFO", "logger": "live_trading", "msg": "Order placed", "symbol": "BTCUSD", "or
# ↑ ts จะเป็นเวลาที่คุณรันจริง – ที่เหลือตรงกันทุกตัว
```

35.2 Kill Switch ด้วย asyncio.Event

```
import asyncio
import signal
import logging

logger = logging.getLogger("live_trading")

class KillSwitch:
    """
    Kill switch สำหรับหยุดระบบ live trading
    รองรับ: Ctrl+C, SIGTERM (Docker stop), programmatic (drawdown exceeded)
    """

    def __init__(self):
        self._event = asyncio.Event()
        self._reason: str = ""

    def trigger(self, reason: str = "manual"):
        """ทริกเกอร์ kill switch"""
        self._reason = reason
        self._event.set()
        logger.critical(f"KILL SWITCH TRIGGERED: {reason}")

    async def wait(self):
        """รอนจนกว่า kill switch จะทำงาน"""
        await self._event.wait()

    @property
    def is_set(self) -> bool:
        return self._event.is_set()

    @property
    def reason(self) -> str:
        return self._reason

    def setup_signal_handlers(self, loop: asyncio.AbstractEventLoop):
        """ตั้งค่า OS signal handlers"""
        def _handle_signal(sig_name: str):
            logger.warning(f"Received {sig_name}")
            self.trigger(f"OS signal: {sig_name}")

        # add_signal_handler รองรับเฉพาะ Unix/Linux/macOS (ไม่ทำงานบน Windows)
        if sys.platform != "win32":
            for sig in (signal.SIGINT, signal.SIGTERM):
                loop.add_signal_handler(
                    sig,
                    lambda s=sig: _handle_signal(s.name)
                )
        else:
            # Windows: ใช้ signal.signal() แทน (synchronous)
            import signal as _signal
            _signal.signal(_signal.SIGINT,
                           lambda s, f: _handle_signal("SIGINT"))
```

35.3 Health Check Loop

```
import asyncio
import time
from dataclasses import dataclass, field
```



```

@dataclass
class SystemHealth:
    last_binance_tick: float = 0.0 # timestamp ของ tick ล่าสุดจาก Binance
    last_bybit_tick: float = 0.0
    last_order_time: float = 0.0
    total_pnl: float = 0.0
    drawdown_pct: float = 0.0
    peak_equity: float = 0.0
    current_equity: float = 0.0
    order_errors: int = 0
    feed_disconnects: int = 0

    def update_equity(self, equity: float):
        if equity > self.peak_equity:
            self.peak_equity = equity
        self.current_equity = equity
        if self.peak_equity > 0:
            self.drawdown_pct = (equity - self.peak_equity) / self.peak_equity

async def health_check_loop(health: SystemHealth,
                             kill_switch: KillSwitch,
                             check_interval: float = 5.0,
                             max_drawdown: float = -0.05,
                             max_feed_silence: float = 30.0):
    """
    ตรวจสอบภาพระบบทุก check_interval วินาที
    หยุดอัตโนมัติถ้า:
    - drawdown เกิน max_drawdown
    - ไม่ได้รับ feed นานกว่า max_feed_silence วินาที
    """
    logger = logging.getLogger("live_trading.health")

    while not kill_switch.is_set:
        await asyncio.sleep(check_interval)
        now = time.time()

        # ตรวจสอบ drawdown
        if health.drawdown_pct < max_drawdown:
            kill_switch.trigger(
                f"Max drawdown exceeded: {health.drawdown_pct:.2%} < {max_drawdown:.2%}"
            )
            break

        # ตรวจสอบ feed silence
        binance_silence = now - health.last_binance_tick
        bybit_silence = now - health.last_bybit_tick

        if health.last_binance_tick > 0 and binance_silence > max_feed_silence:
            logger.error(f"Binance feed silent for {binance_silence:.0f}s")
            kill_switch.trigger(f"Binance feed silent {binance_silence:.0f}s")
            break

        if health.last_bybit_tick > 0 and bybit_silence > max_feed_silence:
            logger.error(f"Bybit feed silent for {bybit_silence:.0f}s")
            kill_switch.trigger(f"Bybit feed silent {bybit_silence:.0f}s")
            break

        # ตรวจสอบ order errors
        if health.order_errors >= 5:
            kill_switch.trigger(
                f"Too many order errors: {health.order_errors}"
            )
            break

        logger.info(
            f"Health OK | equity={health.current_equity:,.0f} "
            f"drawdown={health.drawdown_pct:.2%} "
            f"order_errors={health.order_errors}"
        )

```

35.4 ระบบ Live Trading ครบวงจร

Running Example: BTC Pair Live Trading System

ประกอบทุก component จาก บท 31-34 เป็น production-ready system สมบูรณ์

 **โครงก่อนโค้ด — บล็อกปิดเล่ม ~220 บรรทัด**

นี่คือที่ทุกอย่างมาบรรจบ · **ไม่มีแนวคิดใหม่เลย** — ทุกชิ้นมาจากบทก่อนหน้านี้ งานของบล็อกนี้คือ "ต่อสายให้ถูก"

Binance WS ─┐
└─> asyncio.Queue → _signal_loop → _execute_signal → Bybit REST
Bybit WS ─┐ (บท 33) (บท 32) (บท 34)
 └─ health_check_loop ─┐ kill switch (บท 35)

เมธอด	หน้าที่	มาจากบท	ระดับ
<code>_on_binance_candle</code> <code>_on_bybit_candle</code>	callback ที่ WS client เรียกเมื่อมีแท่งใหม่ — แคโยนเข้า queue	33	●
<code>_compute_zscore</code>	คำนวณ z-score ของ spread จากราคาสองตลาด	13-14	●
<code>_get_signal</code>	แปลง z-score เป็น LONG / SHORT / EXIT / HOLD	14	●
<code>_execute_signal</code>	ส่งออเดอร์จริงผ่าน REST client	34	●
<code>_signal_loop</code>	หัวใจ — ดึงจาก queue → คำนวณ → สั่ง วนไปจนกว่า kill switch ทำงาน	32	●
<code>_shutdown</code>	ปิดสถานะที่ค้าง ยกเลิกออเดอร์ปิด connection	35	●
<code>run</code>	สตาร์ททุก task พร้อมกันด้วย gather แล้วรอ kill switch	32	●

⚠️ **อ่านตามลำดับข้อมูลไหล ไม่ใช่ตามลำดับบรรทัด:** เริ่มที่ `run()` (ล่างสุด) เพื่อเห็นภาพรวมว่าใครถูกสตาร์ทบ้าง → แล้วค่อยตาม `_signal_loop` ขึ้นไปหา `_execute_signal`

```
import asyncio
import logging
import os
import time
import uuid
from collections import deque
from typing import Optional

# =====
# Configuration — โหลดจาก env var เสมอ
# =====

CONFIG = {
    "symbol": "BTCUSDT",
    "interval": "1", # 1 minute candles
    "pair_window": 60, # lookback candles
    "entry_zscore": 2.0,
    "exit_zscore": 0.5,
    "order_qty": 0.001, # BTC per trade
    "max_drawdown": -0.05, # หยุดถ้า drawdown เกิน 5%
    "max_feed_silence": 30.0, # หยุดถ้าไม่มี feed 30s
    "initial_capital": 10_000.0, # USD
    "testnet": True, # เปลี่ยนเป็น False เมื่อพร้อม live
    "log_level": "INFO",
    "log_file": "btc_pair_live.log",
}

class BTCPairLiveSystem:
    """
    Live trading system สำหรับ BTC pair strategy
    Binance/Bybit spread → Z-score → market order
    """
```

```

"""

def __init__(self, config: dict):
    self.config = config
    self.logger = setup_logging(
        config["log_level"], config.get("log_file")
    )
    self.kill_switch = KillSwitch()
    self.health = SystemHealth(
        peak_equity = config["initial_capital"],
        current_equity = config["initial_capital"],
    )
    self.order_tracker = OrderTracker()

    # Price buffers
    w = config["pair_window"]
    self._binance_closes: deque[float] = deque(maxlen=w)
    self._bybit_closes: deque[float] = deque(maxlen=w)

    # Position state
    self._position: Optional[str] = None # "LONG", "SHORT", หรือ None
    self._shared_queue: asyncio.Queue[Candle] = asyncio.Queue(maxsize=1000)

# --- Callbacks ---

def _on_binance_candle(self, candle: Candle):
    self.health.last_binance_tick = time.time()

def _on_bybit_candle(self, candle: Candle):
    self.health.last_bybit_tick = time.time()

# --- Signal Logic ---

def _compute_zscore(self) -> Optional[float]:
    """คำนวณ z-score ของ spread Binance-Bybit"""
    w = self.config["pair_window"]
    if (len(self._binance_closes) < w or
        len(self._bybit_closes) < w):
        return None

    import statistics
    spreads = [
        b - y for b, y in zip(
            list(self._binance_closes),
            list(self._bybit_closes)
        )
    ]
    mean = statistics.mean(spreads)
    stdev = statistics.stdev(spreads)
    if stdev == 0:
        return None
    return (spreads[-1] - mean) / stdev

def _get_signal(self, zscore: float) -> str:
    entry = self.config["entry_zscore"]
    exit_ = self.config["exit_zscore"]

    if zscore > entry: return "SHORT"
    if zscore < -entry: return "LONG"
    if abs(zscore) < exit_: return "EXIT"
    return "HOLD"

# --- Order Execution ---

async def _execute_signal(self, signal: str,
                        client: BybitOrderClient):
    """แปลง signal เป็น order"""
    if signal == "HOLD":
        return
    if signal == self._position:
        return # อยู่ใน position นี้อยู่แล้ว

    qty = self.config["order_qty"]

```

```

sym = self.config["symbol"]
coid = f"PAIR-{uuid.uuid4().hex[:8].upper()}"

if signal in ("LONG", "SHORT"):
    # ปิด position เดิมก่อน (ถ้ามี)
    if self._position is not None:
        close_side = (OrderSide.SELL
                       if self._position == "LONG" else OrderSide.BUY)
        close_coid = f"CLOSE-{uuid.uuid4().hex[:8].upper()}"
        close_order = await client.place_market_order(
            sym, close_side, qty, close_coid
        )
        self.order_tracker.track(close_order)
        if close_order.status == OrderStatus.ERROR:
            self.health.order_errors += 1

    # เปิด position ใหม่
    side = OrderSide.BUY if signal == "LONG" else OrderSide.SELL
    order = await client.place_market_order(sym, side, qty, coid)
    self.order_tracker.track(order)

    if order.status in (OrderStatus.OPEN, OrderStatus.FILLED):
        self._position = signal
        self.logger.info(
            f"Position opened: {signal}",
            extra={"symbol": sym, "order_id": coid, "side": side.value}
        )
    else:
        self.health.order_errors += 1
        self.logger.error(
            f"Order failed: {order.reject_reason}",
            extra={"order_id": coid}
        )

elif signal == "EXIT" and self._position is not None:
    side = (OrderSide.SELL
            if self._position == "LONG" else OrderSide.BUY)
    order = await client.place_market_order(sym, side, qty, coid)
    self.order_tracker.track(order)

    if order.status in (OrderStatus.OPEN, OrderStatus.FILLED):
        self._position = None
        self.logger.info(
            "Position closed",
            extra={"symbol": sym, "order_id": coid}
        )
    else:
        self.health.order_errors += 1

# — Main Consumer Loop —

async def _signal_loop(self, client: BybitOrderClient):
    """ประมวลผล candle จาก queue และส่ง order"""
    while not self.kill_switch.is_set:
        try:
            candle = await asyncio.wait_for(
                self._shared_queue.get(), timeout=1.0
            )
        except asyncio.TimeoutError:
            continue

        # เพิ่มเฉพาะ closed candle
        if candle.is_closed:
            if candle.exchange == "binance":
                self._binance_closes.append(candle.close)
            elif candle.exchange == "bybit":
                self._bybit_closes.append(candle.close)

        zscore = self._compute_zscore()
        if zscore is not None:
            signal = self._get_signal(zscore)
            self.logger.debug(
                f"zscore={zscore:.3f} signal={signal}"
            )

```

```

        )
        if not self.kill_switch.is_set:
            await self._execute_signal(signal, client)

    self._shared_queue.task_done()

# ——— Graceful Shutdown ———

async def _shutdown(self, client: BybitOrderClient):
    """ปิด position ทั้งหมดก่อน shutdown"""
    self.logger.warning("Graceful shutdown: ปิด open positions...")

    if self._position is not None:
        side = (OrderSide.SELL
                 if self._position == "LONG" else OrderSide.BUY)
        coid = f"SHUTDOWN-{uuid.uuid4().hex[:8].upper()}"
        order = await client.place_market_order(
            self.config["symbol"], side,
            self.config["order_qty"], coid
        )
        self.order_tracker.track(order)
        self.logger.info(
            f"Shutdown order: {order.status.name}",
            extra={"order_id": coid}
        )
        self._position = None

    self.logger.info(
        f"Shutdown complete | "
        f"total_orders={len(self.order_tracker._orders)} | "
        f"drawdown={self.health.drawdown_pct:.2%}"
    )

# ——— Run ———

async def run(self):
    """เข้าสู่การทำงาน live"""
    loop = asyncio.get_event_loop()
    self.kill_switch.setup_signal_handlers(loop)

    self.logger.info("Starting BTC Pair Live System")
    self.logger.info(f"Config: {self.config}")

    # WebSocket clients
    binance_client = BinanceWebSocketClient(
        symbol = self.config["symbol"],
        interval = f"{self.config['interval']}m",
        on_candle = self._on_binance_candle,
        queue = self._shared_queue,
    )
    bybit_client = BybitWebSocketClient(
        symbol = self.config["symbol"],
        interval = self.config["interval"],
        on_candle = self._on_bybit_candle,
        queue = self._shared_queue,
    )

    async with BybitOrderClient(testnet=self.config["testnet"]) as order_client:
        try:
            await asyncio.gather(
                binance_client.run(),
                bybit_client.run(),
                self._signal_loop(order_client),
                health_check_loop(
                    self.health,
                    self.kill_switch,
                    max_drawdown = self.config["max_drawdown"],
                    max_feed_silence = self.config["max_feed_silence"],
                ),
                self.kill_switch.wait(), # รอจนกว่า kill switch
                return_exceptions=True,
            )
        finally:

```

```

        binance_client.stop()
        bybit_client.stop()
        await self._shutdown(order_client)

# — Entry point —
if __name__ == "__main__":
    system = BTCPairLiveSystem(CONFIG)
    asyncio.run(system.run())

```

🏠 `main.py` — 5 task ที่วิ่งพร้อมกัน และเงื่อนไขจบของแต่ละตัว

```

await asyncio.gather(
    binance_client.run(),      # ① รับ feed — วนตลอด reconnect เอง
    bybit_client.run(),       # ②
    self._signal_loop(...),   # ③ ประมวลผล — วนจนกว่า kill switch
    health_check_loop(...),   # ④ เช็คสุขภาพ — ทริกเกอร์ kill switch ได้
    self.kill_switch.wait(),   # ⑤ นั่งรอเฉย ๆ — ตัวที่จบก่อนเพื่อน
    return_exceptions=True,
)

```

1. ตัวแรกไม่มีตัวไหนจบเอง — ①② วน reconnect ตลอด, ③④ วนจนกว่า kill_switch ถูกตั้ง ถ้ามีแค่ 4 ตัวนี้ โปรแกรมจะไม่มีทางออกอย่างสวยงาม
2. ⑤ `kill_switch.wait()` คือ "สวิตช์ออก" — มันแค่รอรอพอมีใครสั่ง `trigger()` (จาก Ctrl+C, จาก drawdown เกิน, จาก feed เจียนานเกิน) มันจะตื่นและจบ
3. `return_exceptions=True` สำคัญมากตรงนี้ — ถ้าไม่ใส่ พอ task ใดพัง exception จะเด้งออกจาก `gather` ทันที แต่ task ที่เหลือยังวิ่งอยู่เบื้องหลัง (ตามที่เตือนไว้ในบทที่ 31) ผลคือ `finally` จะทำงานทั้งที่ feed ยังไหลอยู่ — ปิดระบบครึ่ง ๆ กลาง ๆ ใส True แล้ว `gather` จะเก็บ exception เป็นค่าคืนแทน ทำให้เราคุมลำดับปิดได้เอง
4. `finally` ทำงานเสมอ — สั่ง `stop()` ทั้งสอง feed แล้วเรียก `_shutdown()` เพื่อปิดสถานะที่ค้าง · นี่คือส่วนที่แยกระบบเล่นกับระบบที่กล้าใส่เงินจริง

อ่านทั้งบล็อกเป็นประโยคเดียว: "เปิดทุกอย่างพร้อมกัน แล้วนั่งรอสวิตช์ปิด — พอสวิตช์ถูกกด ไม่ว่าด้วยเหตุใด ให้เก็บกวาดให้เรียบร้อยเสมอ"

🔵 [รอบสอง] ทำไม `if __name__ == "__main__":` ถึงสำคัญกว่าที่คิดในไฟล์นี้

ถ้าไม่มีบรรทัดนี้ การ `import` ไฟล์นี้เข้าไปในไฟล์อื่นจะเริ่มเทรดจริงทันที — รวมถึงตอนที่ `pytest` เก็บไฟล์ไปรัน หรือตอนที่ IDE ทำ auto-import เพื่อ auto-complete

เป็นอุบัติเหตุที่เกิดขึ้นจริงกับคนที่เขียน bot: เขียน test แล้ว `pytest` import โมดูล → ระบบ live เริ่มทำงานจากในเครื่อง dev · กฎ: ไฟล์ใดที่ทำอะไร มีผลจริง (ส่งออเดอร์ · เขียน DB · จ่ายเงิน) การกระทำนั้นต้องอยู่ใต้ `if __name__ == "__main__":` เสมอ

💡 สังเกตด้วยว่าโค้ดนี้ไม่มีวันจบเอง — ถ้ารันจริงมันจะค้างรอ kill switch ตลอด นั่นคือพฤติกรรมที่ถูกต้องของระบบ live · ตอนทดสอบ ให้ตั้ง timeout ครอบไว้ (เช่น `asyncio.wait_for(system.run(), timeout=60)`) ไม่งั้นทดสอบจะไม่วันจบ

35.5 Pre-Flight Checklist ก่อน Go Live

```

import asyncio
import os
import aiohttp

async def preflight_check(config: dict) -> bool:
    """
    ตรวจสอบความพร้อมก่อนเริ่ม live trading
    คืน True เฉพาะเมื่อผ่านทุก check
    """
    results = {}
    logger = logging.getLogger("live_trading.preflight")

    # 1. ตรวจสอบ API Keys
    api_key = os.environ.get("BYBIT_API_KEY", "")
    api_secret = os.environ.get("BYBIT_API_SECRET", "")
    results["api_keys"] = bool(api_key and api_secret and
                               len(api_key) > 10 and
                               len(api_secret) > 10)

    if not results["api_keys"]:

```

```

logger.error("FAIL: BYBIT_API_KEY หรือ BYBIT_API_SECRET ไม่ได้ตั้งค่า")

# 2. ตรวจสอบการเชื่อมต่อ exchange
try:
    async with aiohttp.ClientSession() as session:
        url = ("https://api-testnet.bybit.com/v5/market/time"
               if config.get("testnet")
               else "https://api.bybit.com/v5/market/time")
        async with session.get(url, timeout=aiohttp.ClientTimeout(total=5)) as r:
            data = await r.json()
            results["connectivity"] = data.get("retCode") == 0
except Exception as e:
    results["connectivity"] = False
    logger.error(f"FAIL: ไม่สามารถเชื่อมต่อ exchange: {e}")

# 3. ตรวจสอบ balance
if results.get("api_keys") and results.get("connectivity"):
    try:
        async with BybitOrderClient(testnet=config.get("testnet", True)) as client:
            resp = await client._request("GET", "/v5/account/wallet-balance",
                                          {"accountType": "UNIFIED"})

            if resp.get("retCode") == 0:
                coins = resp["result"]["list"][0]["coin"]
                usdt = next(
                    (float(c["availableToWithdraw"])
                     for c in coins if c["coin"] == "USDT"),
                    0.0
                )
                min_balance = config.get("initial_capital", 1000) * 0.5
                results["balance"] = usdt >= min_balance
                if not results["balance"]:
                    logger.error(
                        f"FAIL: USDT balance {usdt:.2f} < required {min_balance:.2f}"
                    )
                else:
                    logger.info(f"OK: USDT balance = {usdt:,.2f}")
            else:
                results["balance"] = False
    except Exception as e:
        results["balance"] = False
        logger.error(f"FAIL: ดึง balance ไม่ได้: {e}")

# 4. ตรวจสอบ config
cfg_ok = (
    config.get("order_qty", 0) > 0 and
    config.get("max_drawdown", 0) < 0 and
    config.get("entry_zscore", 0) > config.get("exit_zscore", 0) > 0
)
results["config"] = cfg_ok
if not cfg_ok:
    logger.error("FAIL: Config ไม่ถูกต้อง")

# 5. สรุป
all_pass = all(results.values())
print("\n=== Pre-Flight Check ===")
for name, ok in results.items():
    status = "✓ PASS" if ok else "✗ FAIL"
    print(f" {name:20s}: {status}")
print(f"\n{'→ READY TO LAUNCH' if all_pass else '→ FIX ISSUES FIRST'}\n")
return all_pass

# asyncio.run(preflight_check(CONFIG))

```

Deployment Checklist — ก่อน Go Live จริง

- ทดสอบบน Testnet ≥ 1 สัปดาห์ — ให้ระบบวิ่งตลอดเวลา ดู drawdown และ error logs
- Paper trading บน real feed — รับ feed จริงแต่ไม่ส่ง order จริง เพื่อตรวจสอบ latency
- ตั้ง max position size เล็กๆ ตอนเริ่มต้น เช่น 0.001 BTC ก่อน scale ขึ้น
- มี monitoring ภายนอก — อย่าเชื่อแค่ log ของตัวเอง ใช้ Telegram bot หรือ Grafana alert
- Deploy บน VPS ใกล้ exchange — Singapore หรือ Tokyo เพื่อ latency ต่ำ

- ตั้ง **systemd** หรือ **Docker** ให้ restart อัตโนมัติถ้า process crash
- **Log rotation** — log file โตได้มาก ตั้ง logrotate หรือ size limit

► แบบฝึกหัด: เพิ่ม Telegram alert เมื่อ kill switch ทำงาน หรือเมื่อ fill order ใหญ่

จบหนังสือ — เส้นทางต่อไป

การเดินทางจาก PART 0 ถึง PART VI

คุณผ่านการเรียนรู้มาไกลมาก — ตั้งแต่ Python พื้นฐานจนถึงระบบ live trading อัตโนมัติ
สรุปทุกอย่างที่ได้เรียนในหนังสือเล่มนี้:

Part	เนื้อหา	สิ่งที่ได้
Part 0	Python Foundations	Syntax, data types, functions, comprehensions
Part I	Data Wrangling	pandas, numpy, matplotlib — ดึง/ทำความสะอาด/visualize ข้อมูลตลาด
Part II	Math Tools	Rolling stats, z-score, cointegration, pair trading signal
Part III	OOP	Classes, ABC, Strategy/Observer/Factory patterns — system ที่ขยายได้
Part IV	Backtesting	Walk-forward, slippage, commission, overfitting, Sharpe/drawdown
Part V	AI-Assisted Coding	Prompt engineering, code generation, audit checklist, quant libraries, full strategy with AI
Part VI	Async & Live Trading	asyncio, WebSocket, order execution, kill switch, monitoring

เส้นทางการพัฒนาในฐานะ Quant Developer: _____ [หนังสือนี้] Part 0-I → Part II-III → Part IV-V → Part VI Python basics Math + OOP Backtest Live System | | | ▼▼▼ [ขั้นต่อไป]
NumPy/Pandas statsmodels vectorbt/ production เชี่ยวชาญ PyMC (Bayes) zipline deployment Nautilus cloud infra

แนะนำขั้นต่อไปหลังจบหนังสือ

1. Backtesting เชิงลึก

- **vectorbt** — vectorized backtesting เร็วกว่า pandas loop 100x
- **Nautilus Trader** — professional backtester + live trading framework ที่ใช้ใน production
- **Zipline-reloaded** — backtester ของ Quantopian ที่ยังมี community ดูแล

2. Statistical Methods เพิ่มเติม

- **Bayesian Inference** ด้วย PyMC — ประเมิน parameter uncertainty แทน point estimate
- **Kalman Filter** — dynamic hedge ratio ที่ปรับตามเวลา (statsmodels มี DLM)
- **Regime Detection** ด้วย Hidden Markov Model — ตรวจสอบว่าตลาดอยู่ใน trend/mean-revert/volatile

3. Machine Learning for Finance

- **Feature Engineering** — microstructure features, order book imbalance, funding rate
- **Gradient Boosting** (XGBoost/LightGBM) สำหรับ signal classification
- **LSTM / Transformer** สำหรับ time series — แต่ระวัง overfitting
- **Reinforcement Learning** (Stable-Baselines3) — ให้ model เรียนรู้ position sizing

4. Infrastructure & Operations

- **Docker + docker-compose** — containerize trading system
- **TimescaleDB** — PostgreSQL extension สำหรับ time series data ใน production
- **Grafana + Prometheus** — monitoring dashboard สำหรับ live system
- **Redis Streams** — message queue สำหรับ high-throughput tick data

5. หนังสือและแหล่งเรียนรู้แนะนำ

- *Advances in Financial Machine Learning* — Marcos Lopez de Prado (อ่านหลังจากมีพื้นฐาน ML)
- *Algorithmic Trading* — Ernest Chan (practical, Python examples)
- *Active Portfolio Management* — Grinold & Kahn (classic quant text)
- QuantLib Python — pricing library สำหรับ options และ derivatives
- Quantopian Lectures (archive) — beginner-friendly quant tutorials

คำเตือนสุดท้าย — จากนักพัฒมาถึงนักพัฒนา

หนังสือนี้สอนวิธี สร้าง trading system — ไม่ใช่วิธีรวยจากตลาด

- Strategy ที่ backtest ได้ดีมากไม่ได้ผลใน live — นั่นคือความจริงของ quant
- Market เปลี่ยนไปตลอด — strategy ที่วันนี้อาจไม่ดีพรุ่งนี้
- การ risk management และ position sizing สำคัญกว่า entry signal มาก
- ไม่มี strategy ไหนที่ไม่มีความเสี่ยง — จัดการความเสี่ยงด้วยระบบ ไม่ใช่ด้วยความหวัง

จงสร้างระบบที่ *fail safely*, *log everything*, และ *never risk what you can't afford to lose*

■ สารบัญทั้งหมด · 🔗 คำศัพท์ Quant Trading